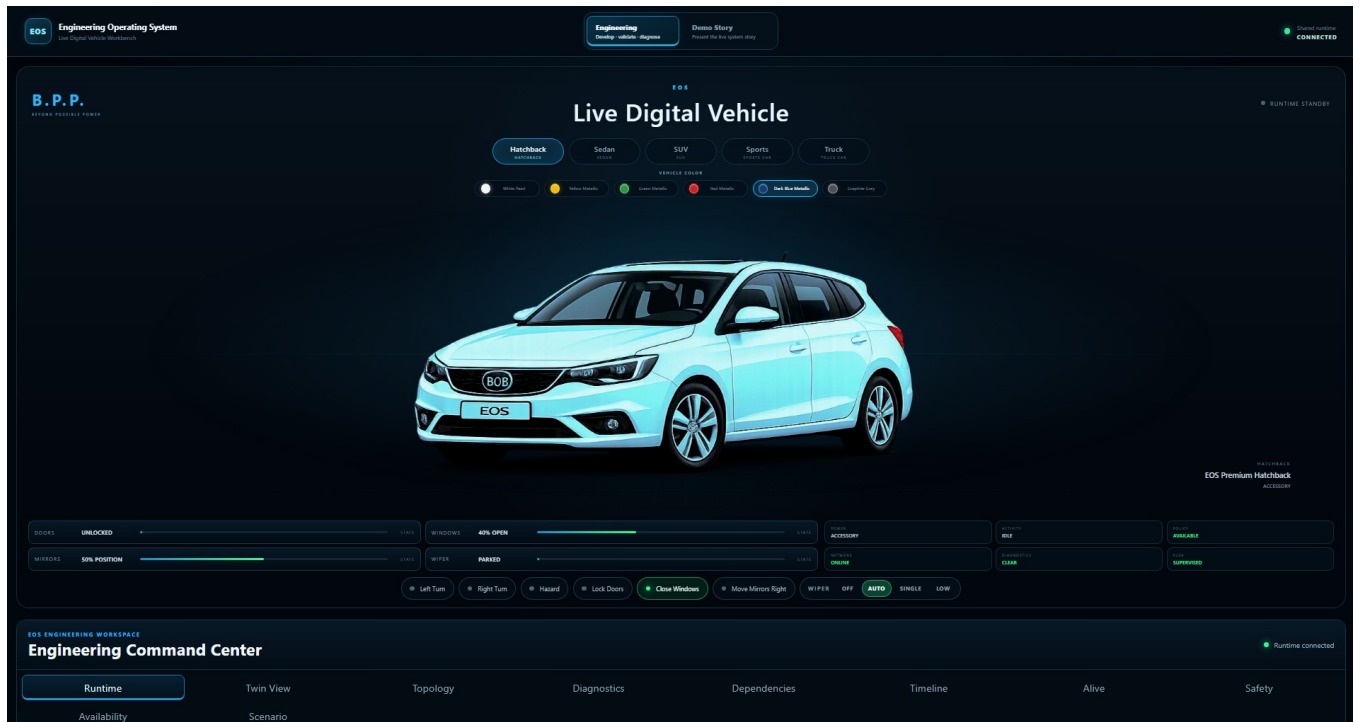


BCM Emulator V1.0

User Guide

Public Preview | Complete visual edition



Try the live Public Preview

[Open BCM Emulator Public Preview](#)

Release Candidate aligned: src(20260926-085701).zip



Table of Contents

1. Introduction
2. Quick Start
3. Understanding the BCM Emulator
4. User Interface Overview
5. Vehicle Configuration
6. BCM Features
7. Runtime and System States
8. Fault Injection and Recovery
9. Working with Scenarios
10. Feature Scenarios
11. Runtime and Recovery Scenarios
12. Functional Safety Scenarios
13. EOS Integrated Mission
14. Demo Story
15. Engineering Observability
16. Reset and Reconstruction
17. Typical User Workflows
18. BCM Emulator V1.0 Scope and Limitations
19. Troubleshooting
20. Appendix

1. Introduction

1.1 Purpose of the BCM Emulator

The **BCM Emulator V1.0** is a browser-based engineering platform for exploring, demonstrating, and validating software-defined vehicle body-control concepts.

It goes beyond a simple graphical vehicle simulator. The platform combines functional body-control features with the supporting runtime architecture required to operate them as part of an integrated software system.

The current V1.0 platform includes six primary body-control features:

- Door Lock
- Fuel Flap
- Window
- Mirror
- Turn Light / Hazard
- Wiper

These features operate on top of a common engineering architecture that includes runtime services, component dependencies, feature lifecycle management, communication, diagnostics, deployment/topology information, fault supervision, recovery mechanisms, and functional-safety behavior.

The purpose of the BCM Emulator is therefore not only to demonstrate **what a vehicle feature does**, but also to make visible **how the surrounding software system behaves while that feature operates**.

For example, a user can observe what happens when:

- a vehicle function is activated;
- a required service becomes unavailable;
- an ECU or HPC node goes offline;
- communication is interrupted;
- a dependency becomes unhealthy;
- the system enters a degraded state;
- a safety-related failure triggers a Safe State;
- the affected component recovers;
- the runtime is reset and reconstructed.

This makes the BCM Emulator useful both as an interactive demonstrator and as an engineering-analysis environment.

1.2 What the Public Preview Demonstrates

The BCM Emulator V1.0 Public Preview is intended to demonstrate an executable, integrated SDV engineering system rather than a collection of isolated feature simulations.

A user can interact with the vehicle functions while simultaneously observing multiple architectural layers of the system:

Vehicle behavior

→ Door Lock, Fuel Flap, Window, Mirror, signaling and Wiper behavior

Feature runtime

→ capability state and operational state

Dependencies

→ required services, providers, activation, health and impact

Deployment

→ software components and their relationship to vehicle computing nodes

Communication

→ communication paths, network state and runtime message activity

Diagnostics

→ active faults and diagnostic evidence

Functional Safety

→ NORMAL, DEGRADED and SAFE STATE behavior

Recovery

→ technical recovery, controlled restoration and Safe-State recovery

Engineering observability

→ Twin View, Dependency View, Event Timeline and other runtime information

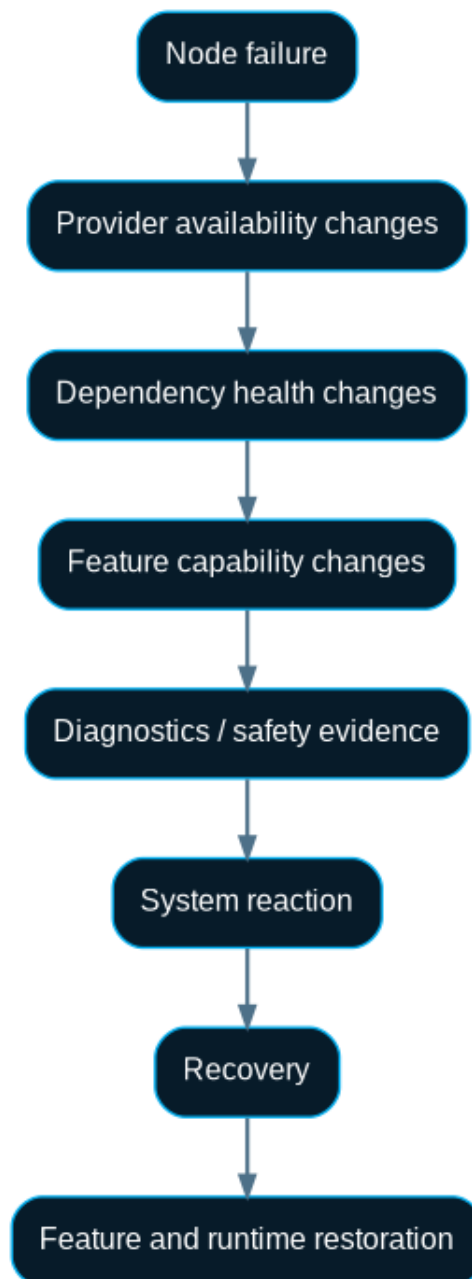
Executable scenarios

→ repeatable demonstrations of normal operation, faults, degradation, recovery, reset and system-level behavior

One of the central ideas of the platform is that these views are not independent visual demonstrations. They represent different perspectives on the **same running system**.

```
Node failure
↓
Provider availability changes
↓
Dependency health changes
↓
Feature capability changes
↓
Diagnostics / safety evidence
↓
System reaction
↓
Recovery
↓
Feature and runtime restoration
```

The corresponding state changes can be observed through the different engineering views.



System reaction chain from node failure through recovery

1.3 Intended Users

The BCM Emulator is primarily intended for users interested in automotive software engineering and software-defined vehicle architecture.

Typical users may include:

- embedded and automotive software engineers;
- software architects;
- system architects;
- functional-safety engineers;
- platform and middleware engineers;
- vehicle-function developers;
- engineering managers and technical leads;
- researchers and students studying SDV architecture;
- technical decision-makers evaluating engineering approaches.

The platform can be used at different levels.

A user interested mainly in vehicle behavior can operate the body-control functions and run demonstration scenarios.

An engineer can go further and inspect lifecycle state, runtime dependencies, topology, diagnostics, communication and safety reactions.

An architect can use the same system to study how these different concerns interact.

The User Guide therefore begins with practical operation and progressively introduces deeper engineering concepts.

1.4 What BCM Emulator V1.0 Includes

The V1.0 Release Candidate contains six integrated feature domains:

Feature	Typical behavior
Door Lock	Lock / unlock with discrete actuator-state transitions
Fuel Flap	Open / close with discrete actuator-state transitions
Window	Continuous actuator movement and positioning
Mirror	Continuous actuator adjustment
Turn Light / Hazard	Signaling, arbitration and coordinated lamp behavior
Wiper	Periodic motion, operating modes and sensor-dependent behavior

These features are integrated with shared platform capabilities including:

- feature lifecycle management;
- operational-state management;
- canonical dependency handling;
- runtime service lifecycle;
- component/node deployment;
- network and communication modeling;
- diagnostics;
- fault injection;
- recovery;
- reset and state reconstruction;
- Functional Safety supervision and reactions;
- engineering observability;
- executable scenario validation.

The V1.0 platform also provides a substantial set of predefined executable scenarios for exploring feature behavior and system reactions.

These scenarios are described later in this guide with concrete execution steps and expected observations.

1.5 What BCM Emulator V1.0 Is Not

The BCM Emulator is an **engineering emulator and reference platform**.

It should not be confused with production vehicle ECU software.

The current implementation runs as a browser-based engineering environment and is designed to make software architecture, runtime behavior and engineering evidence visible and reproducible.

It is therefore not:

- production ECU firmware;
- a replacement for vehicle-level validation;
- a hardware-in-the-loop environment;

- a production communication stack;
- an ISO 26262 certification artifact;
- a claim that the demonstrated architecture has been validated for a specific production vehicle.

Instead, the platform provides an executable environment in which relevant architectural concepts and system behaviors can be designed, exercised and observed.

This distinction is especially important for Functional Safety.

The emulator implements and demonstrates safety-oriented architectural concepts such as fault detection, reaction selection, degraded operation, Safe State, recovery supervision and controlled restoration.

These mechanisms are intended for **engineering exploration and architectural validation within the emulator**, not as evidence of product-level safety certification.

1.6 V1.0 Scope Boundaries

BCM Emulator V1.0 intentionally freezes a defined feature and architecture scope.

One example is runtime dependency binding.

The architecture separates:

```

Dependency Requirement
  ↓
Provider Binding
  ↓
Runtime Health
  ↓
Consumer Impact

```

V1.0 uses **static provider binding**.

The architecture has been structured so that future mechanisms such as primary/fallback providers, dynamic discovery or mode-dependent rebinding can be introduced later, but these mechanisms are not part of the V1.0 runtime behavior.

Likewise, V1.0 should be understood as a stable engineering baseline rather than the final endpoint of the BCM platform.

Future versions may extend:

- the vehicle feature portfolio;
- deployment configurations;
- provider-binding strategies;
- vehicle visualization;
- engineering analysis capabilities;
- validation coverage;
- production-oriented embedded implementations.

The purpose of freezing V1.0 is to establish a **known, reproducible and validated baseline** from which such evolution can proceed.

1.7 How to Use This Guide

This guide is organized so that a new user does not need to understand the complete architecture before using the emulator.

A recommended learning path is:

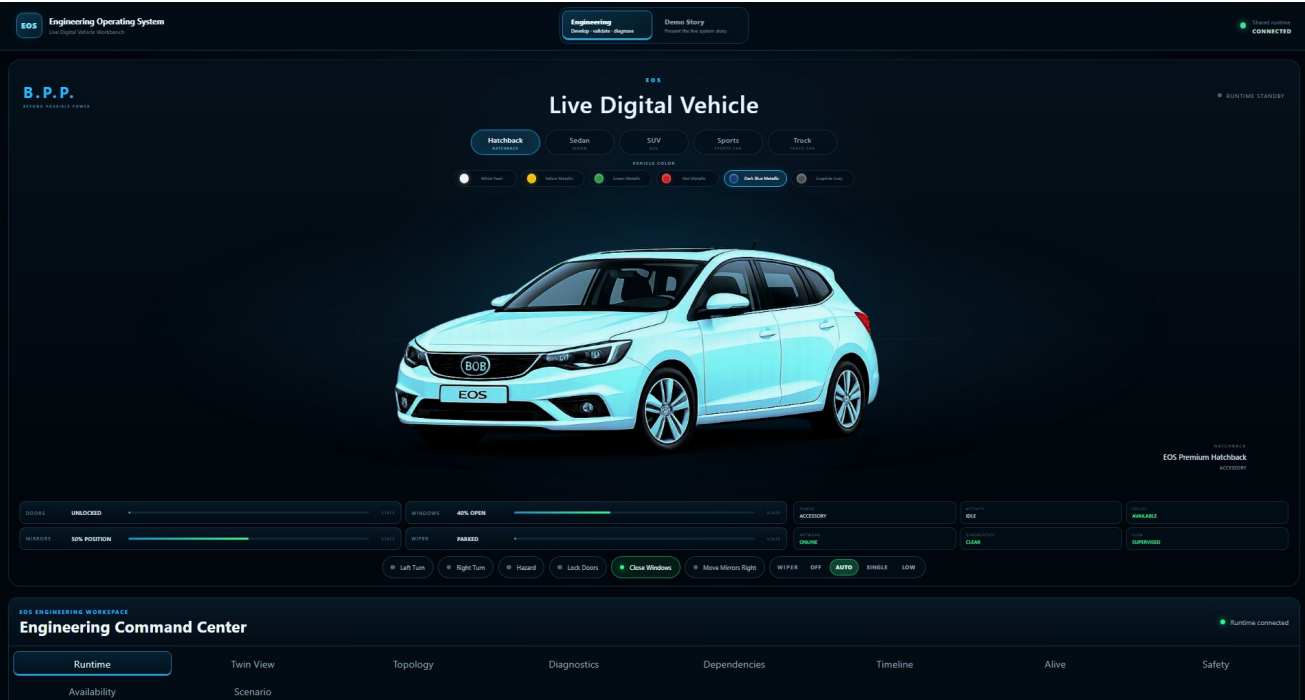
```

Quick Start
  ↓
Explore one vehicle feature
  ↓
Run a simple scenario
  ↓
Observe Runtime / Twin View
  ↓
Explore Dependency and Safety views
  ↓
Run fault and recovery scenarios
  ↓
Run EOS Integrated Mission
  ↓
Use the platform for engineering investigation

```

Users who primarily want to experience the platform can begin directly with **Section 2 — Quick Start**.

Users interested in the architectural concepts can continue through the later sections on runtime states, dependencies, fault handling, Functional Safety, reset/reconstruction and engineering observability.



Main BCM Emulator Engineering workspace

2. Quick Start

This section provides a short first-use path through BCM Emulator V1.0. It is intended to let a new user interact with the platform and run an executable engineering scenario before learning the deeper architecture.

The exact Public Preview URL will be inserted when the Public Preview deployment URL is finalized.

Public Preview URL: _To be inserted when the deployment URL is finalized._

2.1 Launching the BCM Emulator

Open the BCM Emulator V1.0 in a supported desktop browser.

After startup, the application opens in the **Engineering** workspace. The upper workbench bar identifies the product as:

EOS — Engineering Operating System
Live Digital Vehicle Workbench

The workbench provides two primary workspaces:

Workspace	Purpose
Engineering	Develop · validate · diagnose
Demo Story	Present the live system story

The upper-right runtime indicator should show:

Shared runtime — CONNECTED

The Engineering workspace is the recommended starting point for a first-time user.

The vehicle initially starts with its vehicle power state at **OFF**. The Live Digital Vehicle therefore appears in runtime standby until the vehicle is powered.

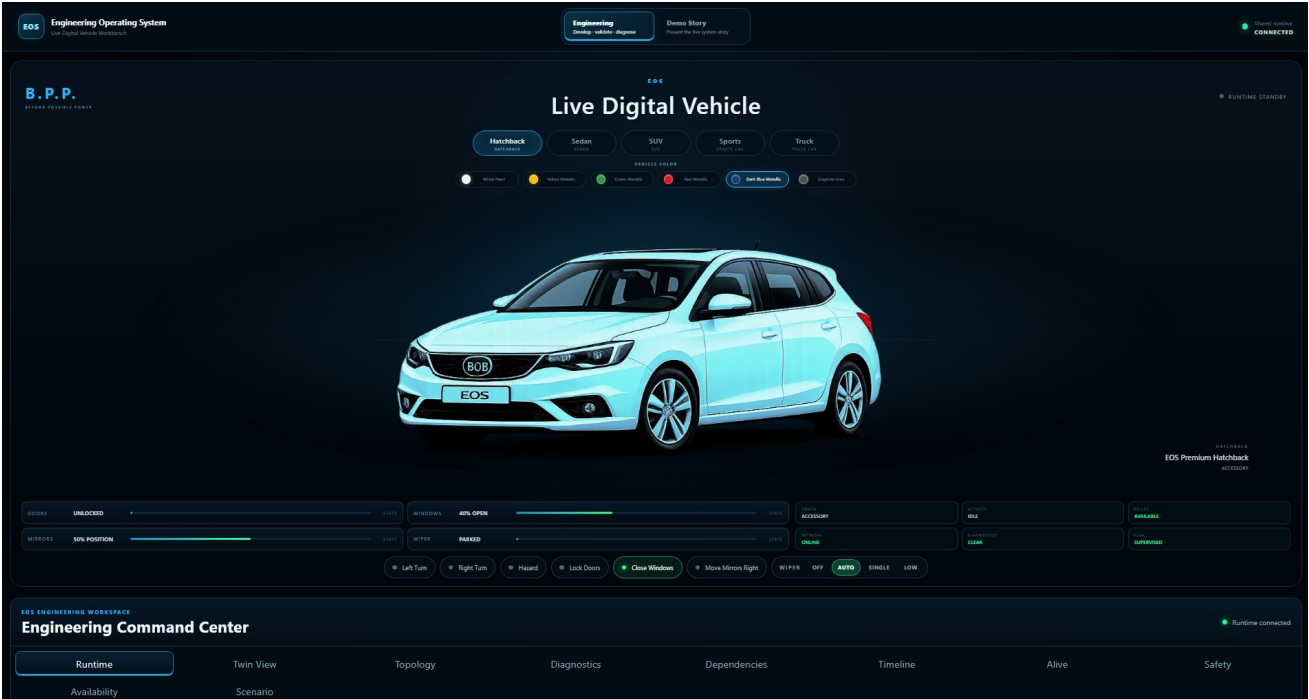


Figure 2-1 - Main BCM Emulator screen after startup

2.2 Basic UI Orientation

The Engineering workspace contains two major areas.

Live Digital Vehicle

The upper vehicle area provides the interactive vehicle representation and direct feature controls.

It includes:

- vehicle-package selection;
- vehicle-color selection;
- Live Digital Vehicle visualization;
- runtime/engineering status information;
- Left Turn and Right Turn controls;

- Hazard control;
- Door Lock / Unlock;
- Window control;
- Mirror control;
- Wiper modes.

The Wiper control provides the modes:

- OFF
- AUTO
- SINGLE
- LOW

Depending on the current state of a feature, the text of a feature control may change. For example, the Door Lock control changes between **Lock Doors** and **Unlock Doors**, while the Window control changes between **Open Windows**, **Close Windows**, and **Stop Windows**.

Engineering Command Center

Below the vehicle area, the **Engineering Command Center** provides the engineering views of the same running system. The V1.0 Engineering views are:

View	Main purpose
Runtime	Overall operating picture and runtime controls
Twin View	Synchronized engineering representation of current runtime state
Topology	Vehicle computing nodes, hosted components and communication topology
Diagnostics	Diagnostic history and active diagnostic evidence
Dependencies	Runtime dependency requirements, providers, health and impact
Timeline	Engineering event timeline
Alive	Alive/liveness supervision information
Safety	Functional-safety state and supervision
Availability	Service, feature, node and runtime-component availability
Scenario	Executable scenario validation and step-by-step results

The **Runtime** view is selected by default and is the best entry point for a new user.

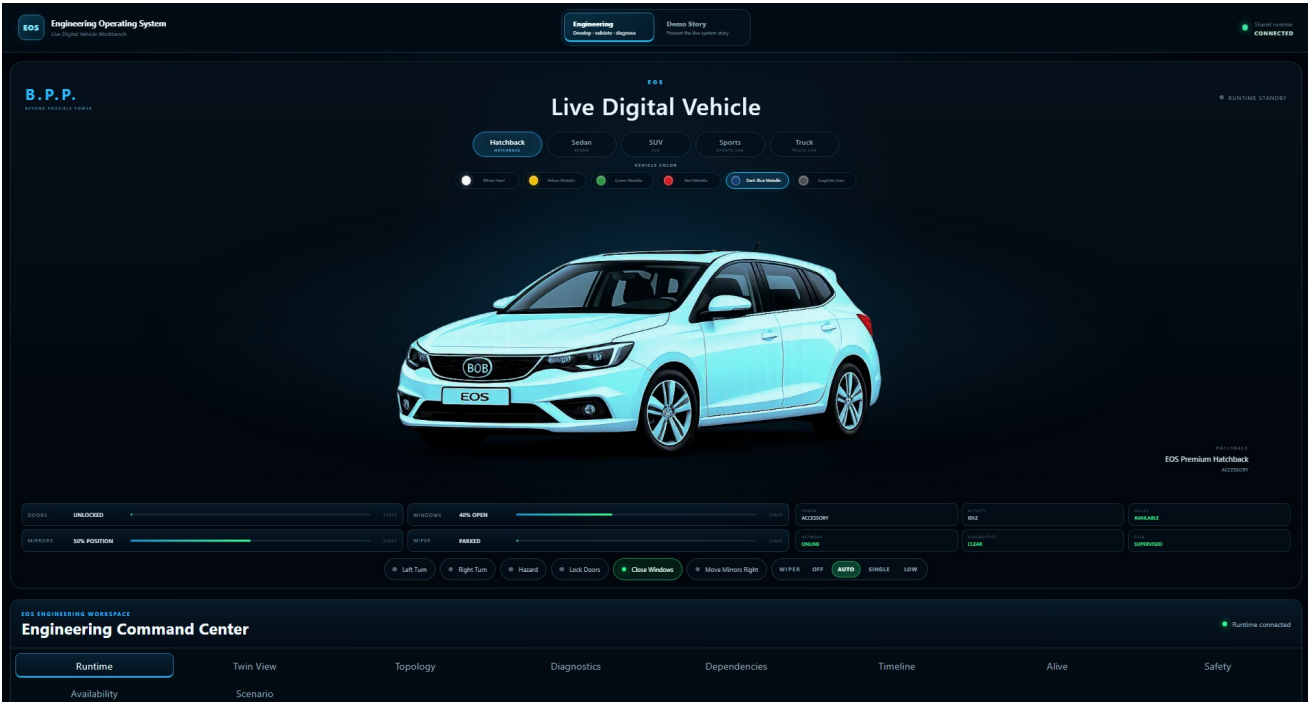


Figure 2-2 - Engineering workspace and Engineering Command Center

2.3 First Interaction — Power the Vehicle and Operate a Feature

The following short exercise verifies that the user interface and shared runtime are working together.

Step 1 — Select the Engineering workspace

At the top of the application, select **Engineering** if it is not already active.

Step 2 — Open the Runtime view

In the Engineering Command Center, select **Runtime**.

The Runtime view contains the **Runtime control** card. The card provides vehicle-power controls, quick feature commands, and scenario execution.

Step 3 — Set the vehicle to IGNITION

In:

Runtime control → **Vehicle state**

select:

IGNITION

The vehicle runtime should transition from standby to an active powered state.

The Live Digital Vehicle header should indicate:

RUNTIME ONLINE

and the current vehicle state should be reflected in the Runtime view.

Step 4 — Toggle Hazard

In:

Runtime control → **Quick commands**

select:

Hazard

Alternatively, the **Hazard** control below the Live Digital Vehicle can be used.

The vehicle visualization should show coordinated indicator activity. The Runtime view should also reflect the active body-control request.

Select **Hazard** again to switch it off.

Step 5 — Try one additional feature

For example, select:

Lock doors

The Door Lock state should change through the corresponding actuator/domain transition and settle in the locked state.

The same runtime can now be inspected through other engineering views such as **Twin View**, **Timeline**, **Availability**, or **Diagnostics**.

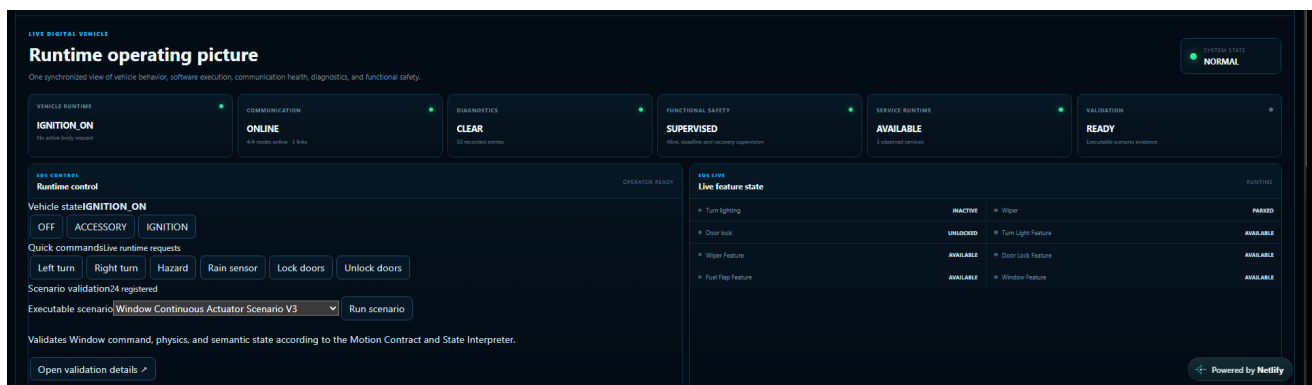


Figure 2-3 - Runtime control with vehicle state, quick commands and scenario validation

2.4 Run Your First Engineering Scenario

Executable scenarios are one of the main ways to explore BCM Emulator V1.0.

For a first run, use a relatively simple feature scenario:

DoorLock Discrete Actuator Scenario V1

Step 1 — Locate Scenario Validation

Stay in:

Engineering → **Runtime**

and locate the **Scenario validation** section inside the Runtime control card.

Step 2 — Select the scenario

Under **Executable scenario**, select:

DoorLock Discrete Actuator Scenario V1

The scenario description is shown below the selector.

Step 3 — Run the scenario

Select:

Run scenario

While it executes, the Runtime control card reports that a scenario is running.

BCM Emulator automatically establishes the canonical scenario baseline before executing the scenario. The user does not need to perform a manual reset first.

Step 4 — Open validation details

Select:

Open validation details

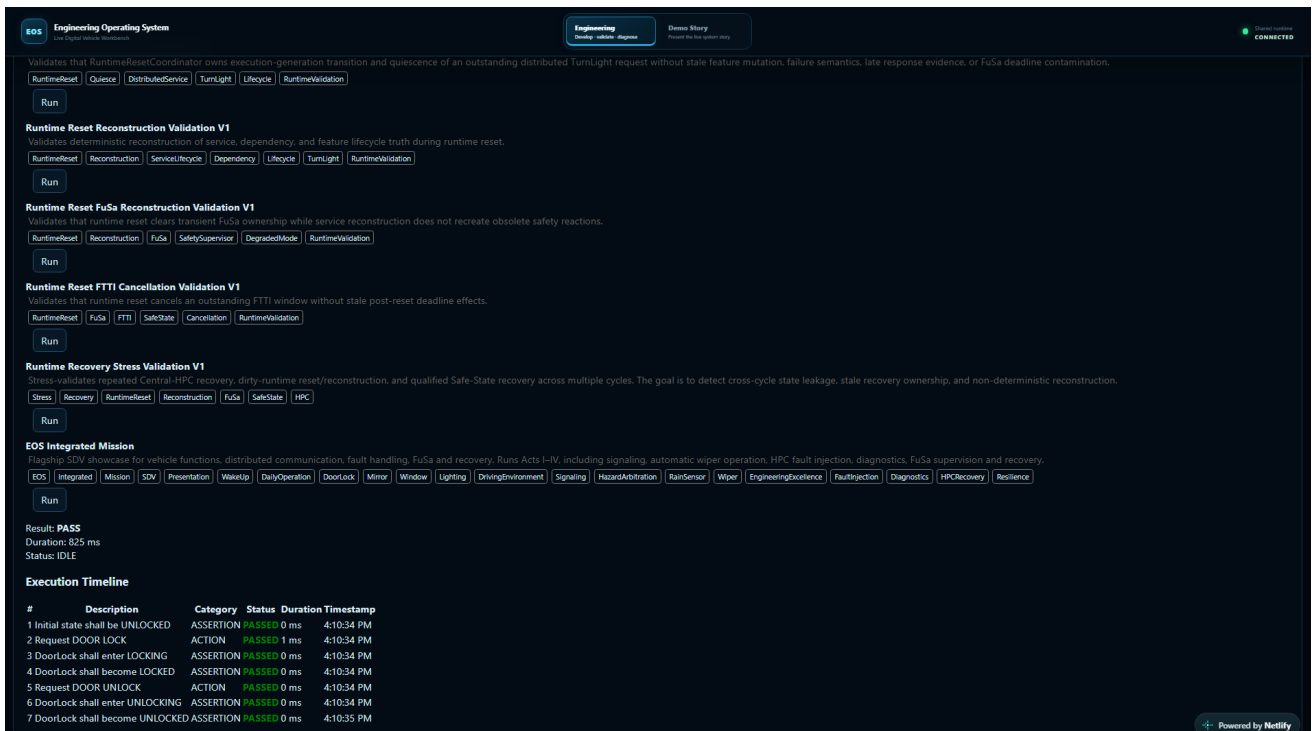
or directly select the **Scenario Engineering** view.

The **Scenario Validation** view shows:

- scenario name;
- scenario description;
- scenario tags;
- execution status;
- final PASS / FAIL result;
- total duration;
- step-by-step execution timeline;
- step category;
- individual step status;
- step duration;
- timestamp.

For a successful execution, the final scenario result should be:

PASS



The screenshot displays the EOS Engineering Operating System interface. At the top, there are tabs for 'Engineering' (selected), 'Runtime', and 'Demos'. Below the tabs, a navigation bar shows 'RuntimeReset', 'Quiesce', 'DistributedService', 'TurnLight', 'Lifecycle', and 'RuntimeValidation'. The main content area shows the 'Runtime Reset Reconstruction Validation V1' scenario. It includes a description: 'Validates deterministic reconstruction of service, dependency, and feature lifecycle truth during runtime reset.' Below the description are buttons for 'RuntimeReset', 'Reconstruction', 'ServiceLifecycle', 'Dependency', 'Lifecycle', 'TurnLight', and 'RuntimeValidation'. A 'Run' button is also present. Below this, there are sections for 'Runtime Reset FuSa Reconstruction Validation V1', 'Runtime Reset FTI Cancellation Validation V1', and 'Runtime Recovery Stress Validation V1', each with their own descriptions and buttons. At the bottom, there is a section for 'EOS Integrated Mission' with a description: 'Hagship SDV showcase for vehicle functions, distributed communication, fault handling, FuSa and recovery, Runs Acts I-IV, including signaling, automatic wiper operation, HPC fault injection, diagnostics, FuSa supervision and recovery.' Below this is a 'Run' button. The final result is displayed as 'Result: PASS', 'Duration: 825 ms', and 'Status: IDLE'. An 'Execution Timeline' table is shown at the bottom.

#	Description	Category	Status	Duration	Timestamp
1	Initial state shall be UNLOCKED	ASSERTION	PASSED	0 ms	4:10:34 PM
2	Request DOOR LOCK	ACTION	PASSED	1 ms	4:10:34 PM
3	DoorLock shall enter LOCKING	ASSERTION	PASSED	0 ms	4:10:34 PM
4	DoorLock shall become LOCKED	ASSERTION	PASSED	0 ms	4:10:34 PM
5	Request DOOR UNLOCK	ACTION	PASSED	0 ms	4:10:34 PM
6	DoorLock shall enter UNLOCKING	ASSERTION	PASSED	0 ms	4:10:34 PM
7	DoorLock shall become UNLOCKED	ASSERTION	PASSED	0 ms	4:10:35 PM

Figure 2-4 - Scenario Validation after a successful execution

2.5 Understanding Scenario Reset Behavior

BCM Emulator V1.0 deliberately does not expose a general-purpose operator button that simply forces the entire runtime into an assumed "healthy" state.

Instead, executable scenarios use the canonical runtime reset/reconstruction mechanism.

Before every scenario execution, the Scenario Runner performs:

```

Scenario selected
  ↓
Canonical runtime reset
  ↓
Runtime reconstruction
  ↓
Known scenario baseline
  ↓
Scenario execution
  ↓
Completed state remains observable

```

This design has two important consequences.

A clean baseline is automatic

A user can run scenarios repeatedly or in a different order without manually preparing the previous scenario state. Each scenario begins from the same canonical runtime baseline.

The completed state is intentionally preserved

After a scenario finishes, BCM Emulator does **not** immediately reset the system again.

The resulting state remains visible so that the user can inspect:

- feature state;
- dependency impact;
- diagnostics;
- safety state;
- topology;
- availability;
- events;
- other engineering evidence.

The next scenario execution owns establishment of the next clean baseline.

Important: Vehicle power **OFF** is a vehicle runtime state; it is not equivalent to a complete BCM runtime reset.

This distinction becomes important in the later sections on Reset and Reconstruction.

2.6 Observe the Same System Through Multiple Views

After the first scenario has completed, keep the resulting runtime state and switch between several Engineering views.

A useful first observation path is:

```

Runtime
  ↓
Twin View
  ↓
Topology
  ↓
Dependencies
  ↓
Timeline
  ↓
Safety
  ↓
Availability

```

The views provide different perspectives on the same shared runtime.

For example:

- **Runtime** summarizes the current operating picture.
- **Twin View** shows synchronized feature/runtime state.
- **Topology** shows where components are deployed and which nodes are online.
- **Dependencies** shows what a selected consumer needs and which provider currently satisfies that requirement.
- **Timeline** shows runtime engineering events.
- **Safety** shows system safety state and safety supervision.

- **Availability** separates service, feature, node and deployed-runtime-component availability.

A central rule when using the emulator is:

Do not interpret one view as an independent simulation. The views are projections of the same running BCM system.

This is why a fault or recovery action may become visible simultaneously in several parts of the Engineering workspace.

2.7 Switch to Demo Story

After becoming familiar with the Engineering workspace, select:

Demo Story

from the upper workbench bar.

Demo Story uses the same shared runtime but presents it differently. It is optimized for explaining the live system story rather than for detailed diagnosis.

Switching between **Engineering** and **Demo Story** does not start a second BCM runtime.

Both workspaces observe and interact with the same runtime state.

This means that a state created in the Engineering workspace can also be observed in Demo Story, and vice versa.

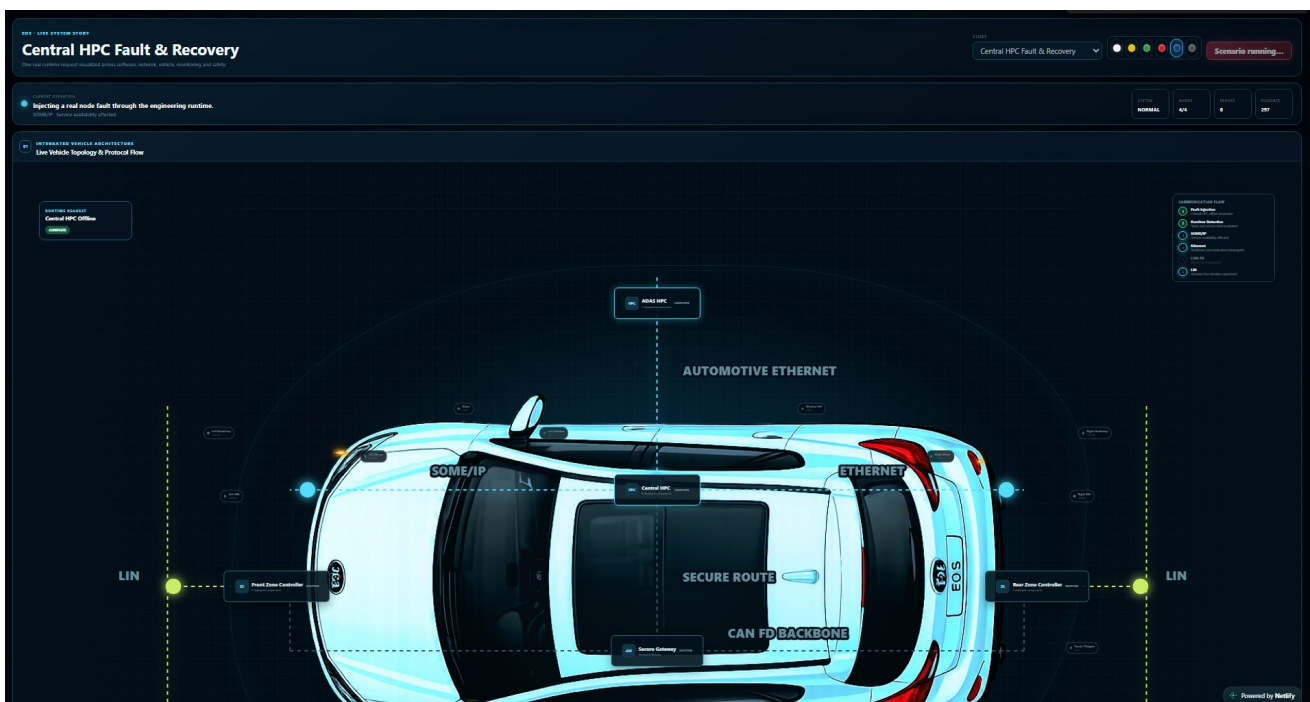


Figure 2-5 - Demo Story using the shared runtime

2.8 Recommended Five-Minute First Session

A new user can obtain a useful overview of BCM Emulator V1.0 with the following short sequence:

1. Open the emulator and confirm **Shared runtime — CONNECTED**.
2. Stay in **Engineering**.
3. Open **Runtime**.
4. Set vehicle state to **IGNITION**.
5. Toggle **Hazard** and observe the vehicle.
6. Lock and unlock the doors.
7. Run **DoorLock Discrete Actuator Scenario V1**.
8. Open **Scenario** and confirm the execution result.
9. Inspect **Twin View**, **Timeline**, **Dependencies**, and **Safety**.
10. Switch to **Demo Story** and confirm that it reflects the same runtime.
11. Return to Engineering and try another scenario.

After these steps, the user has already exercised the three central interaction modes of the platform:

operate → validate → observe

The remaining sections of this guide explain each capability in detail.

3. Understanding the BCM Emulator

Before using the deeper engineering views, it helps to understand one central idea:

BCM Emulator V1.0 is one running system observed through several engineering perspectives.

The vehicle visualization, feature controls, runtime state, dependency information, topology, diagnostics, safety state and event timeline are not separate simulations. They are different views of the same shared runtime.

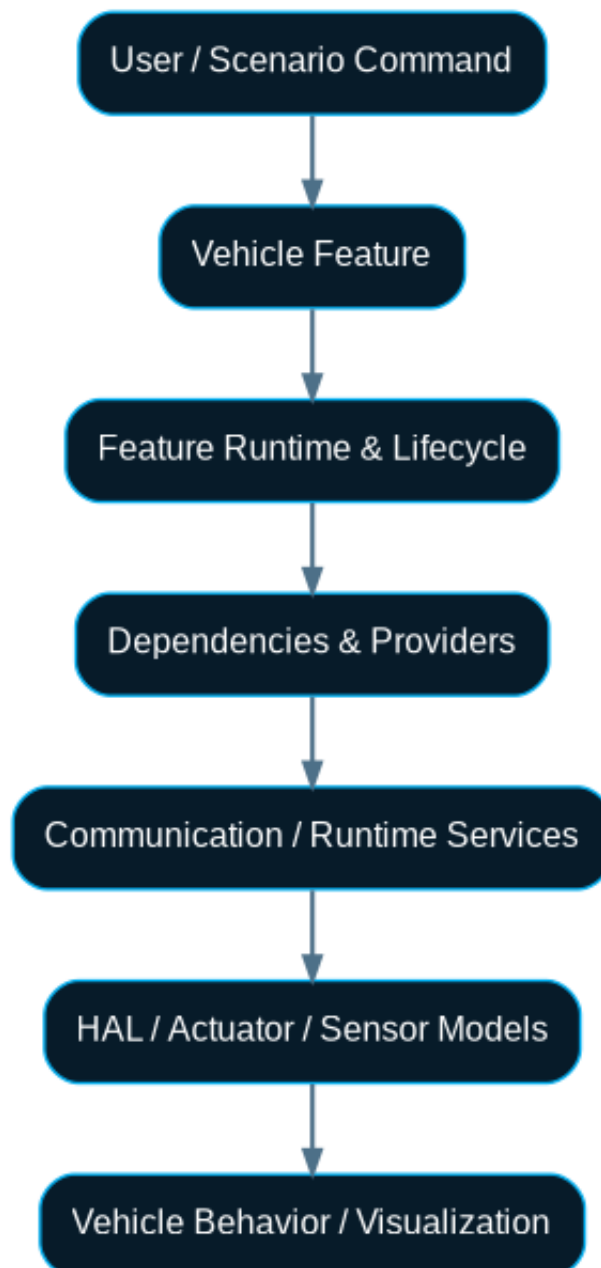
This section introduces the minimum system model needed to interpret those views correctly.

3.1 From Vehicle Behavior to Software Runtime

At the most visible level, BCM Emulator provides familiar body-control functions such as locking doors, moving windows, operating mirrors, signaling and wiping.

Behind those functions is a layered software system.

A useful first mental model is:



The exact execution path depends on the feature, but the important point is that visible vehicle behavior is supported by runtime architecture rather than being drawn directly by the UI.

For example, when a Door Lock request is executed, the user may observe:

```

User command
  ↓
DoorLock feature
  ↓
Actuator command / runtime transition
  ↓
DoorLock domain state changes
  ↓
Vehicle visualization updates

```

At the same time, other engineering views can show feature capability, operational state, events, diagnostics and dependency information.

3.2 Feature, Service, Runtime Component and Node

Several kinds of software entities coexist in the emulator. They should not be treated as interchangeable.

Feature

A **feature** represents vehicle-domain functionality.

Examples include:

- DoorLockFeature
- FuelFlapFeature
- WindowFeature
- MirrorFeature
- TurnLightFeature
- WiperFeature

A feature owns its domain behavior and interprets what that behavior means to the vehicle.

Runtime Service

A **runtime service** provides reusable functionality required by other parts of the system.

A service has its own lifecycle state. In V1.0, service states include:

- REGISTERED
- STARTING
- AVAILABLE
- DEGRADED
- FAILED
- RECOVERING
- UNAVAILABLE

A service being registered does **not** automatically mean it is operational.

Runtime Component

A **runtime component** is a software component that may be deployed to a computing node.

A component can be visible in topology/deployment information even when its state is owned by a mechanism other than the service lifecycle.

This distinction is important because:

```

registered component
  ≠
available service
  ≠
available feature
  ≠
online node

```

Node

A **node** represents a computing location in the emulated vehicle architecture.

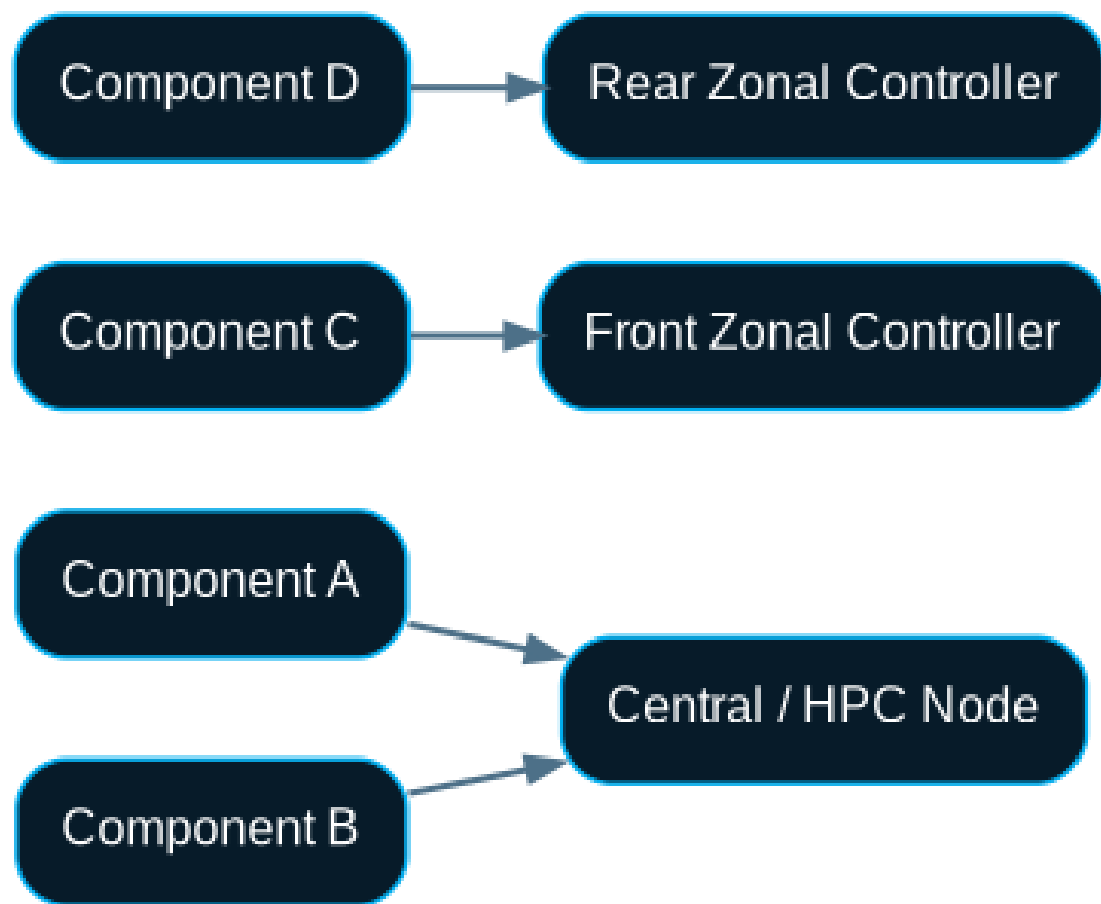
Examples include HPC or zonal-controller nodes.

A component can be deployed to a node. If the node becomes unavailable, components hosted on it may be affected according to their own runtime semantics.

3.3 Deployment: Where Software Runs

BCM Emulator maintains a deployment model that maps software components to vehicle computing nodes.

Conceptually:



This mapping answers:

Where is this component deployed?

It does not by itself answer:

Is the component healthy?

That second question belongs to runtime lifecycle, dependency health, feature capability or other state owners.

This separation is visible in the **Availability** view, which distinguishes:

- Nodes
- Runtime Services
- Deployed Runtime Components
- Runtime Features

This is intentional. V1.0 avoids collapsing all four concepts into one generic "available/unavailable" flag.

3.4 Three Different Feature-State Questions

One of the most important concepts in BCM Emulator is that a feature has more than one kind of state.

For users, three questions should be kept separate:

1. What is the feature doing?

This is the **domain state**.

Examples:

Door Lock

UNLOCKED
LOCKING
LOCKED
UNLOCKING

Wiper

PARKED
wiping / operating modes

Turn Light

```
LEFT
RIGHT
HAZARD
idle
```

Domain states are feature-specific.

2. Can the feature provide its intended capability?

This is the **capability state**.

BCM Emulator V1.0 uses:

Capability state	Meaning
INITIALIZED	Feature exists but usable capability has not yet been established
AVAILABLE	Required conditions permit intended capability
DEGRADED	Feature remains usable with reduced capability
UNAVAILABLE	Intended capability cannot currently be provided
FAILED	Internal feature failure

3. Is the feature currently performing its operation?

This is the **operational state**.

V1.0 uses:

- ACTIVE
- INACTIVE

For example:

```
DoorLock = LOCKED
Capability = AVAILABLE
Operational = INACTIVE
```

is completely valid.

So is:

```
DoorLock = LOCKING
Capability = AVAILABLE
Operational = ACTIVE
```

And a feature can also be:

```
Domain state = stable
Capability = DEGRADED
Operational = INACTIVE
```

The three dimensions must not be interpreted as synonyms.



Key rule: AVAILABLE does not mean ACTIVE, and INACTIVE does not mean unhealthy.

3.5 Dependencies: What a Component Needs

Features and services depend on other components or capabilities.

V1.0 represents these relationships using a canonical dependency model.

A dependency describes:

- the consumer;
- what capability is required;
- the provider;
- whether the requirement is active;
- provider health;
- impact on the consumer;
- recovery policy.

A simplified dependency chain is:

```
Consumer
↓ requires
```

```

Capability
  ↓ provided by
Provider
  ↓
Health evidence
  ↓
Consumer impact

```

Dependency health in V1.0 can be:

- AVAILABLE
- DEGRADED
- UNAVAILABLE
- UNKNOWN

The resulting impact on the consumer is separately represented as:

- NONE
- DEGRADED
- UNAVAILABLE

That separation is important.

A provider can become unhealthy, but the effect on a consumer depends on the dependency contract.

Conditional dependencies

Not every dependency is active all the time.

The Wiper feature provides a useful example.

The Rain Sensor is relevant to **AUTO** operation. If the vehicle is using a manual Wiper mode, the Rain Sensor dependency may be inactive.

Conceptually:

```

Rain Sensor unavailable
  +
Wiper manual mode
  ↓
dependency inactive
  ↓
no dependency impact

```

If the user switches to AUTO:

```

Rain Sensor unavailable
  +
AUTO mode
  ↓
dependency active
  ↓
Wiper capability affected

```

The **Dependency View** lets users inspect these relationships directly.

3.6 Requirement and Provider Are Different Concepts

V1.0 distinguishes between:

Dependency Requirement

What capability the consumer needs.

and:

Provider Binding

Which component currently provides that capability.

In Public Preview V1.0, provider binding is **STATIC**.

This means that a dependency requirement has a predefined provider.

The architecture is structured so that future versions may support strategies such as fallback providers or dynamic discovery, but those mechanisms are outside the V1.0 runtime scope.

For a user, the practical interpretation is simple:

The Dependency View shows the provider currently defined by the V1.0 architecture; the emulator does not dynamically switch to another provider when that provider fails.

3.7 Current State Is Not the Same as History

BCM Emulator records events and diagnostic evidence over time.

This historical information is valuable, but it must not be confused with current runtime truth.

A fundamental V1.0 rule is:

History ≠ current state.

For example, a service may fail and later recover.

The Event Timeline and diagnostic history can continue to show that the failure occurred.

However:

```
historical failure entry
      ≠
currently active fault
```

The current system state must be derived from current authoritative runtime evidence.

This is especially important when interpreting:

- Diagnostics;
- Twin View;
- Safety;
- Availability;
- recovery scenarios.

The system intentionally keeps historical evidence visible after recovery so users can understand what happened without forcing the current system to remain degraded.

3.8 System Safety State

Functional-safety behavior is represented separately from normal feature and service lifecycle.

At system level, BCM Emulator V1.0 uses the canonical safety-state vocabulary:

```
NORMAL
DEGRADED
SAFE STATE
```

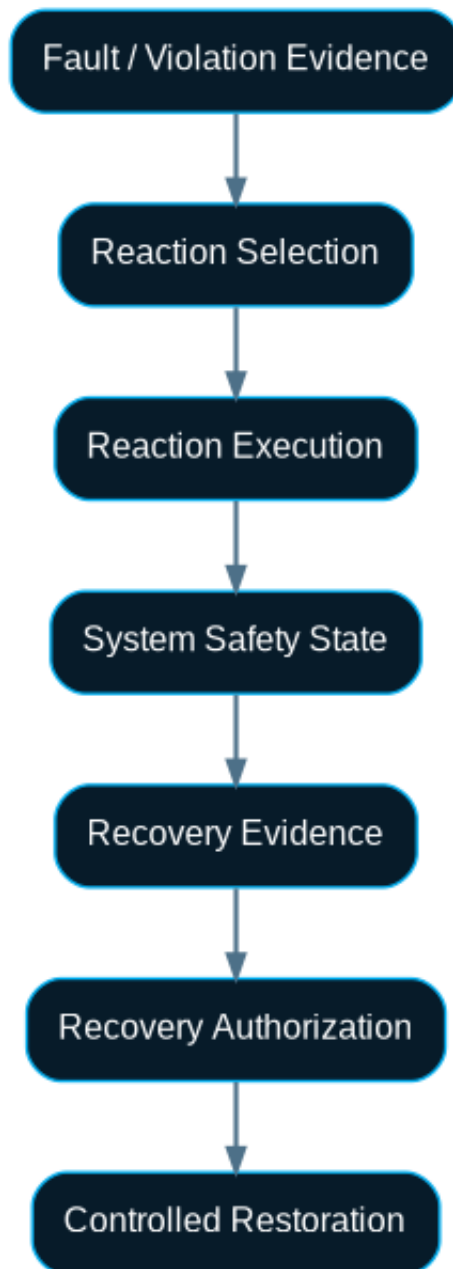
The V1.0 Release Candidate aligns the Safety Control Center, Engineering Control and Demo Story presentation on this same vocabulary.

These states answer a different question from feature capability.

For example:

- a feature can be **DEGRADED** without the complete system entering **SAFE STATE**;
- a runtime service failure can affect dependencies before a safety reaction is selected;
- an active **SAFE STATE** must not be flattened into a generic **DEGRADED** display.

A simplified safety flow is:



For higher-severity safety behavior, recovery from Safe State is intentionally controlled.

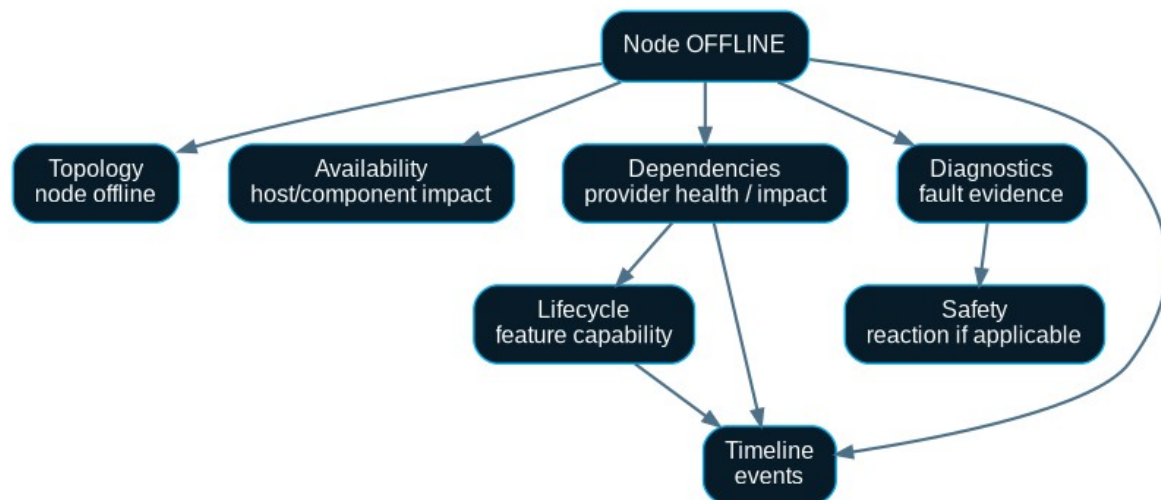
Technical recovery of the failed component alone does not automatically mean that the system may leave Safe State.

The Safety scenarios later in this guide demonstrate this distinction in concrete steps.

3.9 One Event Can Affect Several Views

Because all Engineering views observe the same shared runtime, a single fault may appear differently depending on the view.

For example, when a node goes offline:



Not every event must affect every view.

The purpose of the different views is to answer different engineering questions:

Question	Best starting view
What is the overall runtime doing?	Runtime
What does the synchronized digital representation show?	Twin View
Where is a component deployed?	Topology
What does this consumer depend on?	Dependencies
What is currently available?	Availability
What fault evidence exists?	Diagnostics
What happened over time?	Timeline
Is liveness supervision healthy?	Alive
What is the system safety state?	Safety
Did an executable validation pass?	Scenario

Later sections explain each view in detail.

3.10 A Practical Investigation Pattern

When a user sees unexpected behavior, a useful investigation sequence is:

1. Observe the visible vehicle/feature behavior
↓
2. Check Runtime / Twin View
↓
3. Check Availability
↓
4. Inspect Dependencies
↓
5. Inspect Diagnostics
↓
6. Inspect Safety if relevant
↓
7. Use Timeline to reconstruct what happened

This sequence is not mandatory, but it helps separate:

- visible symptom;
- current runtime state;
- dependency cause;
- diagnostic evidence;
- safety consequence;
- historical sequence.

That separation is one of the main engineering purposes of the BCM Emulator.

3.11 What to Remember Before Continuing

A new user does not need to memorize the complete architecture.

The following rules are sufficient for the rest of this guide:

1. **There is one shared BCM runtime.**
2. **Engineering and Demo Story are different views of that same runtime.**

3. Feature domain state, capability state and operational state are different concepts.
4. A node, runtime service, deployed component and feature have different state ownership.
5. Dependencies describe requirements, providers, health and consumer impact.
6. Historical events do not define current runtime truth.
7. Functional Safety is a separate system-level concern.
8. A fault may be visible in several views because those views observe the same underlying runtime.

With this model in mind, the detailed UI becomes much easier to interpret.

4. User Interface Overview

BCM Emulator V1.0 is organized around a common workbench shell and two primary workspaces:

- **Engineering**
- **Demo Story**

Both workspaces use the same shared runtime.

This section introduces the main areas of the user interface and explains what each view is intended for. Detailed feature operation, fault injection and scenario workflows are covered in later sections.

4.1 Workbench Header

The top workbench bar remains visible while switching between the main workspaces. It contains three important areas.

Product identity

The left side identifies the environment as:

EOS — Engineering Operating System
Live Digital Vehicle Workbench

Workspace switcher

The center area provides the two workspace choices:

Workspace	UI description	Primary use
Engineering	Develop · validate · diagnose	Interactive engineering, diagnosis and validation
Demo Story	Present the live system story	Guided system-level presentation

Shared runtime status

The right side shows the runtime connection state:

Shared runtime — CONNECTED

This status is important because both workspaces operate on the same running BCM instance. Switching workspaces does not create or restart the runtime.

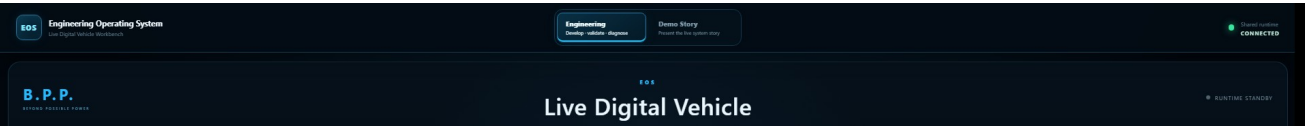
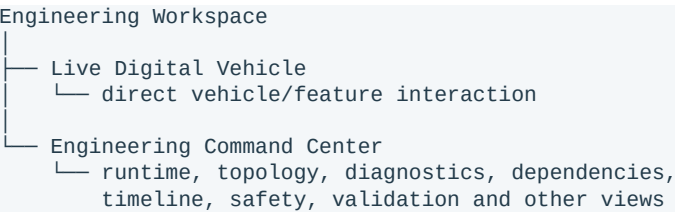


Figure 4-1 - Workbench header and workspace switcher

4.2 Engineering Workspace

The **Engineering** workspace is the primary working environment. It contains two major regions:



The Engineering Command Center also shows:

Runtime connected

when the live runtime is available.

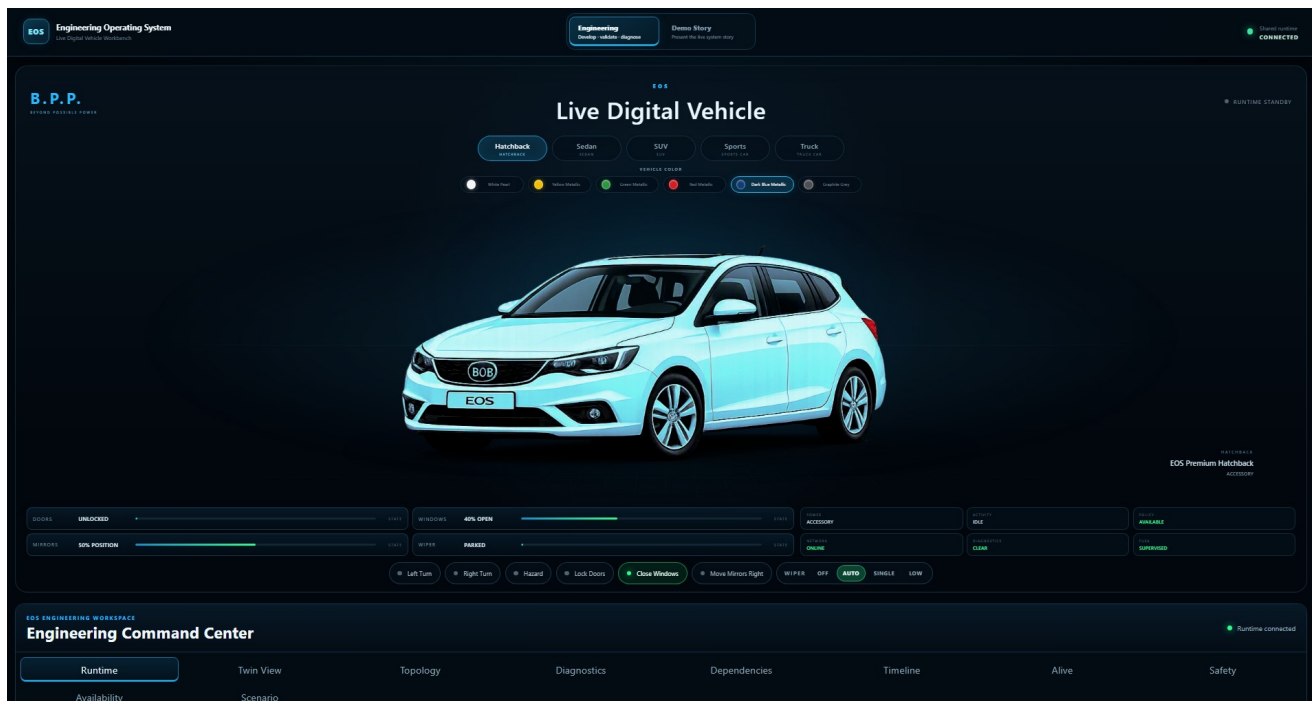


Figure 4-2 - Full Engineering workspace

4.3 Live Digital Vehicle

The Live Digital Vehicle is the interactive visual representation of the vehicle.

It combines:

- vehicle visualization;
- package selection;
- vehicle-color selection;
- current runtime state;
- feature controls;
- runtime/engineering summaries.

The vehicle presentation changes according to live feature state. For example, signaling activity is shown by the lighting overlay.

The feature-control area provides direct controls for:

- **Left Turn**
- **Right Turn**
- **Hazard**
- **Lock Doors / Unlock Doors**
- **Open Windows / Close Windows / Stop Windows**
- **Move Mirrors Left / Move Mirrors Right / Stop Mirrors**
- **Wiper:**
- **OFF**
- **AUTO**
- **SINGLE**
- **LOW**

Control labels may change according to the current state.

For example:

```
Doors unlocked
↓
button = Lock Doors

Doors locked
↓
button = Unlock Doors
```

and:

```
Window stopped + closed
↓
Open Windows

Window moving
↓
```

```

Stop Windows
Window stopped + open
↓
Close Windows

```

The Live Digital Vehicle is useful for direct interaction, but it should not be interpreted as the complete system state. The Engineering views below it provide the software/runtime evidence behind the visible vehicle behavior.

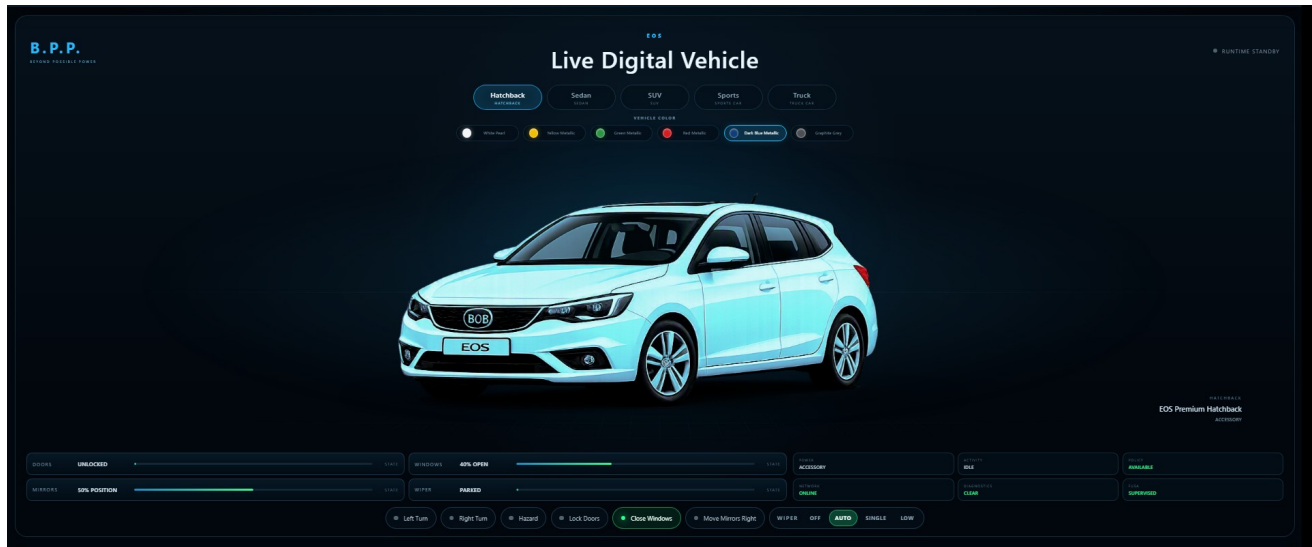


Figure 4-3 - Live Digital Vehicle and feature controls

4.4 Engineering Command Center Navigation

The Engineering Command Center provides ten V1.0 views:

1. Runtime
2. Twin View
3. Topology
4. Diagnostics
5. Dependencies
6. Timeline
7. Alive
8. Safety
9. Availability
10. Scenario

On wider browser layouts, these views appear as tabs.

A view selector is also provided so the same Engineering views remain accessible where the tab layout is less convenient.

The selected Engineering view is preserved while the user works inside the Engineering workspace.

The following subsections summarize what each view is for.

4.5 Runtime View

The **Runtime** view is the default Engineering view.

Its purpose is to provide an overall operating picture of the current BCM system.

The top area summarizes:

- **Vehicle Runtime**
- **Communication**
- **Diagnostics**
- **Functional Safety**
- **Service Runtime**
- **Validation**

The system-state indicator gives a high-level view of the current system condition.

The Runtime view also contains the **Runtime control** card.

Vehicle state

The vehicle power controls are:

- OFF
- ACCESSORY
- IGNITION

Quick commands

The Runtime control provides quick access to:

- Left turn
- Right turn
- Hazard
- Rain sensor / Clear rain
- Lock doors
- Unlock doors

Scenario validation

The same panel allows a registered executable scenario to be selected and started.

It shows:

- number of registered scenarios;
- selected scenario;
- scenario description;
- **Run scenario**;
- current operator/scenario status;
- **Open validation details** .

The Runtime view is therefore the best starting point for:

- bringing the vehicle into a desired power state;
- issuing simple runtime commands;
- starting scenarios;
- checking the overall operating picture before investigating details.

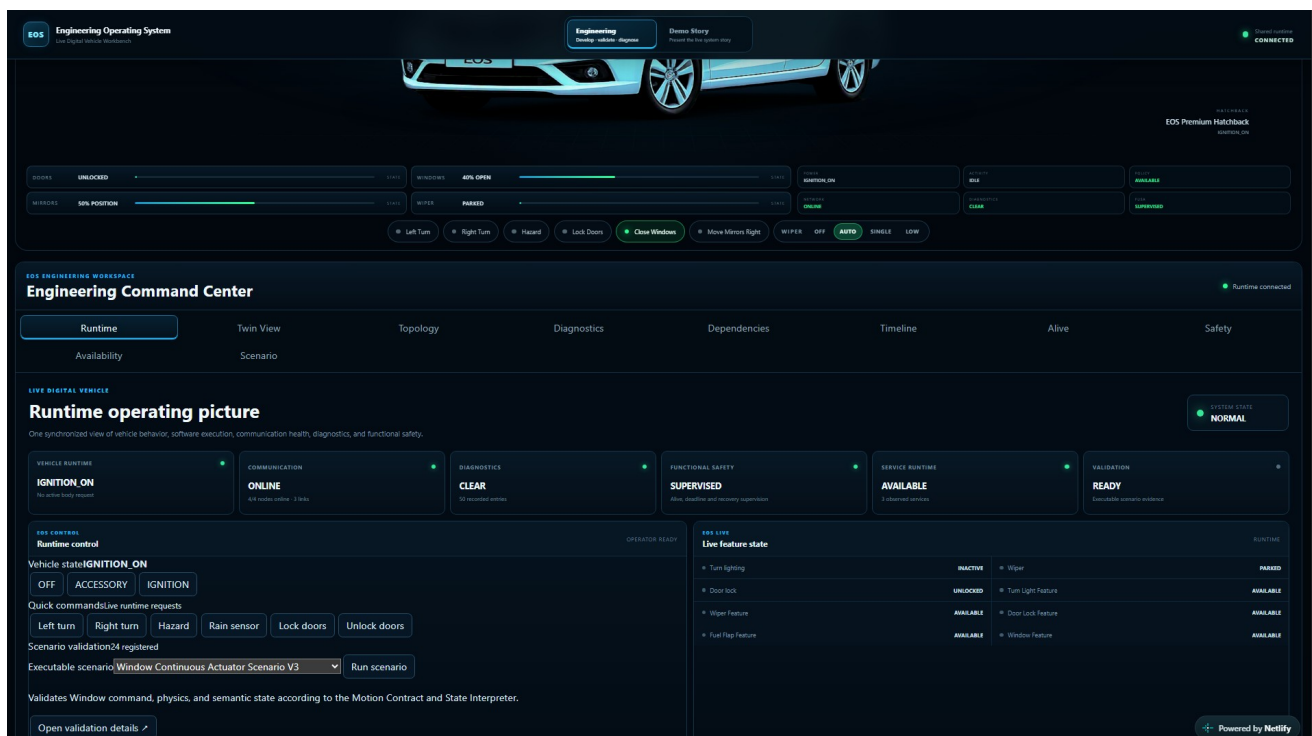


Figure 4-4 - Runtime view with runtime control and operating picture

4.6 Twin View

The **Twin View** presents physical behavior and software truth side by side.

Its central message is:

Physical behavior and software truth — synchronized

The left side represents:

WHAT IS HAPPENING — Physical Vehicle

The right side represents:

WHY IT IS HAPPENING — EOS Runtime Representation

Between them, synchronization information indicates whether the two sides remain aligned.

The view includes:

- physical vehicle behavior;
- power state;
- lighting state;
- Wiper state;
- Door Lock state;
- runtime branches;
- feature/service/safety representation;
- synchronization status;
- live synchronization event stream.

Twin View is especially useful when a user wants to correlate:

visible vehicle behavior
↔
software/runtime state

For example, after triggering signaling or a fault/recovery sequence, the user can see whether the physical representation and runtime representation remain synchronized.

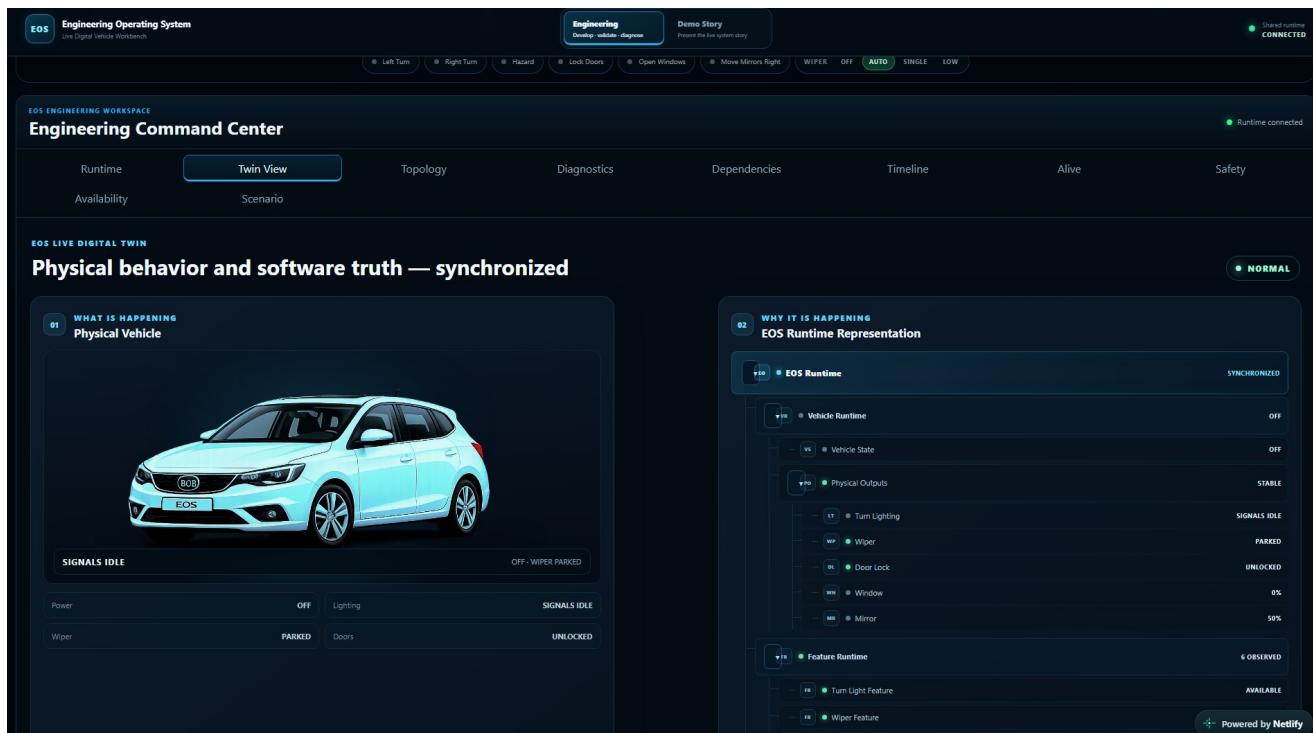


Figure 4-5 - Twin View: physical behavior and runtime truth

4.7 Topology View

The **Topology** view is the live topology and communication explorer.

Its purpose is summarized by the UI as:

See every request move through the vehicle

It provides:

- virtual vehicle computing nodes;
- hosted components;
- network links;
- routed communication activity;
- communication trace;
- runtime execution path;
- network-fault injection.

The topology view can visualize communication and runtime execution across the virtual vehicle architecture.

It can also show node-offline conditions and communication problems.

The **Fault Injection** area is intended for controlled engineering experiments. It should be used when the user intentionally wants to observe how runtime behavior changes under network or node faults.

Topology answers questions such as:

- Which node hosts this component?
- Which node is offline?
- Which communication route is active?
- Where did a request travel?
- Where did an execution path become blocked?

4.8 Diagnostics View

The **Diagnostics** view displays recorded diagnostic evidence.

The table contains:

- Time
- Severity
- Status
- Source
- Message

Diagnostics are historical engineering evidence.

An important interpretation rule is therefore:

A diagnostic entry remaining in the table does not necessarily mean that the fault is still active.

Recovery can change current runtime state while the diagnostic history remains available for analysis.

The **Status** column helps distinguish the diagnostic lifecycle represented by each entry.

For current system truth, use Diagnostics together with views such as:

- Runtime;
- Availability;
- Dependencies;
- Safety;
- Twin View.

EOS Engineering Operating System

Engineering
Topology | Subtree | Diagrams

Demo Story
Present the live system story

Channel: runtime
CONNECTED

AvailabilityScenario

Diagnostics

Time	Severity	Status	Source	Message
4:19:30 PM	INFO	RESOLVED	ServiceRegistry	Service BrakeDecisionService changed state: RECOVERING -> AVAILABLE
4:19:30 PM	INFO	RESOLVED	ServiceRegistry	Service PredictiveLightingService changed state: RECOVERING -> AVAILABLE
4:19:29 PM	INFO	OBSERVATION	ServiceRegistry	Service BrakeDecisionService changed state: FAILED -> RECOVERING
4:19:29 PM	INFO	OBSERVATION	ServiceRegistry	Service PredictiveLightingService changed state: FAILED -> RECOVERING
4:19:29 PM	WARNING	OBSERVATION	PredictiveLightingService	requestPrediction rejected: Service request failed after retries: PredictiveLightingService.requestPrediction
4:19:29 PM	ERROR	OBSERVATION	DeadlineSupervisor	Safety deadline violation: FINAL_FAILURE PredictiveLightingService.requestPrediction
4:19:29 PM	WARNING	OBSERVATION	SafetyMonitorManager	Safety violation counter PredictiveLightingService.requestPrediction = 2
4:19:29 PM	ERROR	OBSERVATION	DeadlineSupervisor	Safety deadline violation: TIMEOUT PredictiveLightingService.requestPrediction
4:19:29 PM	WARNING	OBSERVATION	SafetyMonitorManager	Safety violation counter PredictiveLightingService.requestPrediction = 1
4:19:29 PM	ERROR	ACTIVE	DistributedServiceClient	Timeout calling PredictiveLightingService.requestPrediction (1000 ms)
4:19:28 PM	ERROR	OBSERVATION	NetworkRouter	Message dropped: SERVICE_REQUEST, source=Central HPC, target=ADAS HPC, reason=Fault injection: target node offline
	ERROR	OBSERVATION		Fault reaction selected: SAFE_STATE (SERVICE_STATE_FAILED)
4:19:28 PM	ERROR	OBSERVATION	SafeStateManager	System entered safe state: BrakeDecisionService
4:19:28 PM	ERROR	RESOLVED	ServiceRegistry	Service BrakeDecisionService changed state: AVAILABLE -> FAILED
	WARN	OBSERVATION		Fault reaction selected: DIAGNOSTIC_ONLY (SERVICE_STATE_FAILED)
4:19:28 PM	ERROR	RESOLVED	ServiceRegistry	Service PredictiveLightingService changed state: AVAILABLE -> FAILED
4:19:28 PM	INFO	OBSERVATION	TurnLightFeature	Vehicle power state changed to OFF
4:19:28 PM	INFO	OBSERVATION	DependencySupervisor	WiperFeature dependency 'wiper-actuation' (WiperActuatorModel) => AVAILABLE, active=true, impact=NONE
4:19:28 PM	INFO	OBSERVATION	DependencySupervisor	WiperFeature dependency 'wiper-rain' (RainSensorModel) => AVAILABLE, active=true, impact=NONE
4:19:28 PM	INFO	OBSERVATION	TurnLightFeature	Vehicle power state changed to OFF
4:19:28 PM	INFO	OBSERVATION	ServiceRegistry	Service BrakeDecisionService changed state: STARTING -> AVAILABLE
4:19:28 PM	INFO	OBSERVATION	ServiceRegistry	Service BrakeDecisionService changed state: REGISTERED -> STARTING
4:19:28 PM	INFO	OBSERVATION	ServiceRegistry	Service PredictiveLightingService changed state: STARTING -> AVAILABLE
4:19:28 PM	INFO	OBSERVATION	ServiceRegistry	Service PredictiveLightingService changed state: REGISTERED -> STARTING

Powered by Mettli

Figure 4-7 - Diagnostics evidence during a recovery experiment

4.9 Dependency View

The **Dependencies** view exposes the canonical runtime dependency model.

The view provides two directions:

Depends on

Start from a selected **Consumer** and inspect what it requires.

Conceptually:

```
Consumer
  → Requirement / Capability
  → Provider
```

Used by

Start from a selected **Provider** and inspect which consumers depend on it.

Conceptually:

```
Provider
  ← Requirement
  ← Consumer
```

The view includes filters:

- **Active only**
- **Problems only**

It also summarizes:

- Relations
- Active
- Issues

Selecting a dependency relationship opens detailed information including:

Relationship

- Consumer
- Requirement
- Kind
- Level
- Lifecycle phases

Binding

- Mode
- Provider
- Activation
- conditional activation information where applicable
- current ACTIVE / INACTIVE state

Runtime semantics

- Health source
- Health
- Impact
- reaction on unavailability
- Recovery policy

This is one of the most useful views for understanding *why* a feature becomes degraded or unavailable.

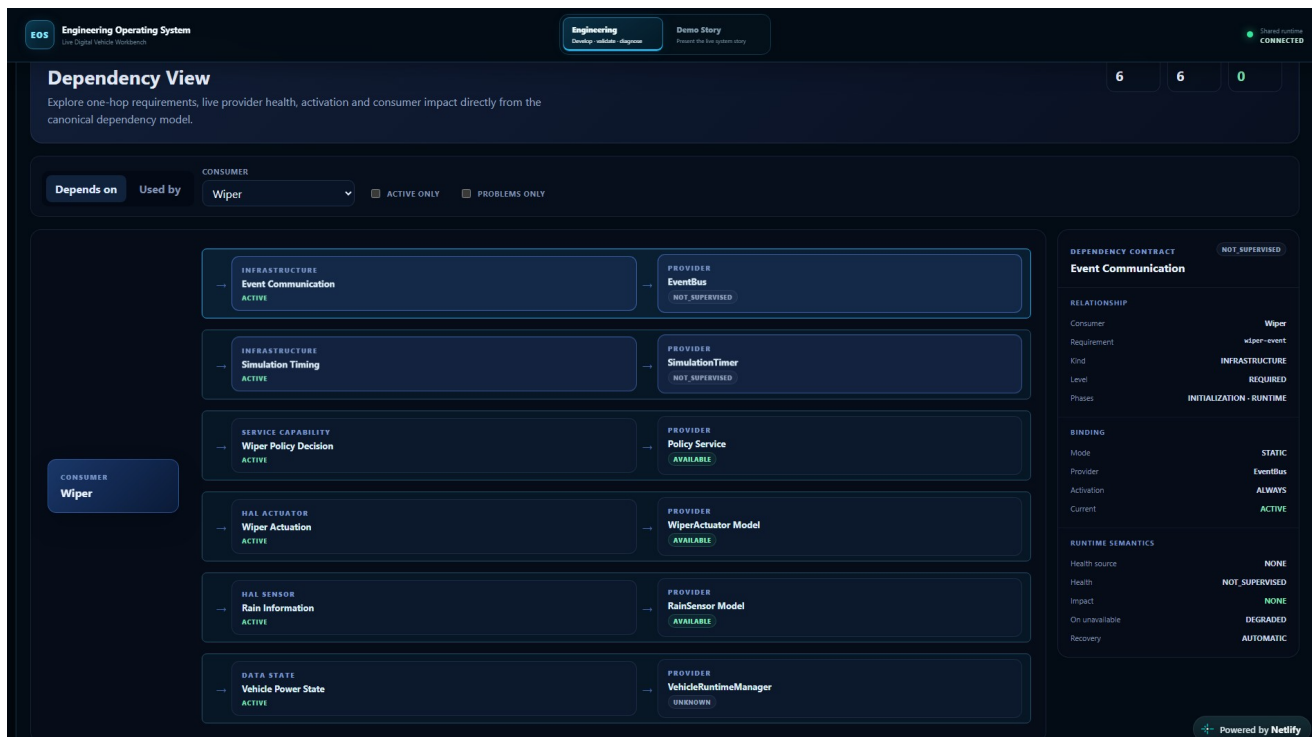


Figure 4-8 - Dependency View in Depends on mode

4.10 Event Timeline

The **Timeline** view provides chronological engineering events emitted by the runtime.

The table contains:

- Time
- Event
- Payload
- Trigger
- Reason
- Feature
- Source
- Target

Events can be filtered by category:

- All
- Input
- Business
- Communication
- Availability
- Safety

Payloads may contain detailed JSON evidence.

The V1.0 layout wraps long payloads while preserving readable columns. On narrow browser windows, the event table can scroll horizontally rather than collapsing important fields into unreadable columns.

The Timeline is particularly useful for reconstructing:

```

what happened
↓
in what order
↓
with what trigger/reason
↓
between which runtime participants
  
```

The Timeline represents historical execution evidence. It should therefore be interpreted together with current-state views rather than used as the sole source of current truth.



Figure 4-9 - Event Timeline with runtime evidence

4.11 Alive View

The **Alive** view displays liveness supervision information.

For each observed component, it shows:

- **Component**
- **Status**
- **Last Alive**
- **Elapsed**

The primary statuses are:

- **ALIVE**
- **UNRESPONSIVE**

This view is useful for inspecting heartbeat/alive supervision independently from higher-level feature or safety state.

For example, a component becoming unresponsive may generate evidence that later affects diagnostics, dependency behavior or Functional Safety.

The Alive view shows the supervision evidence itself.

4.12 Safety View

The **Safety** view is the Functional Safety engineering workspace.

Its title is:

Safety Control Center

and its purpose is:

Runtime supervision, fault reaction and recovery evidence.

The current safety state is presented as one of the system-level conditions such as:

- **NORMAL**
- **DEGRADED**
- **SAFE STATE**

The view summarizes:

- **Safety Supervisor**
- **Recovery**
- **Reaction Target**
- **Reason**

FuSa Fault Injection & Recovery

The Safety view also contains controlled engineering actions.

Fault-injection controls include:

- **Fail PolicyService**
- **Fail BrakeDecisionService**
- **Stop PolicyService Alive**

Recovery/restoration controls include:

- **Restore PolicyService**
- **Restore BrakeDecisionService**
- **Resume PolicyService Alive**
- **Authorize Safe-State Recovery**
- **Request Safe-State Recovery**

These controls inject source-side conditions or recovery requests through runtime engineering interfaces.

They do **not** directly set the system safety state.

This distinction is fundamental:

```

engineering control
  ↓
source/runtime evidence
  ↓
safety evaluation
  ↓
selected reaction
  ↓
system safety state
  
```

The view additionally provides:

- Safety Timeline;
- Safety Evidence statistics;
- component-level safety statistics;
- recent safety diagnostics.

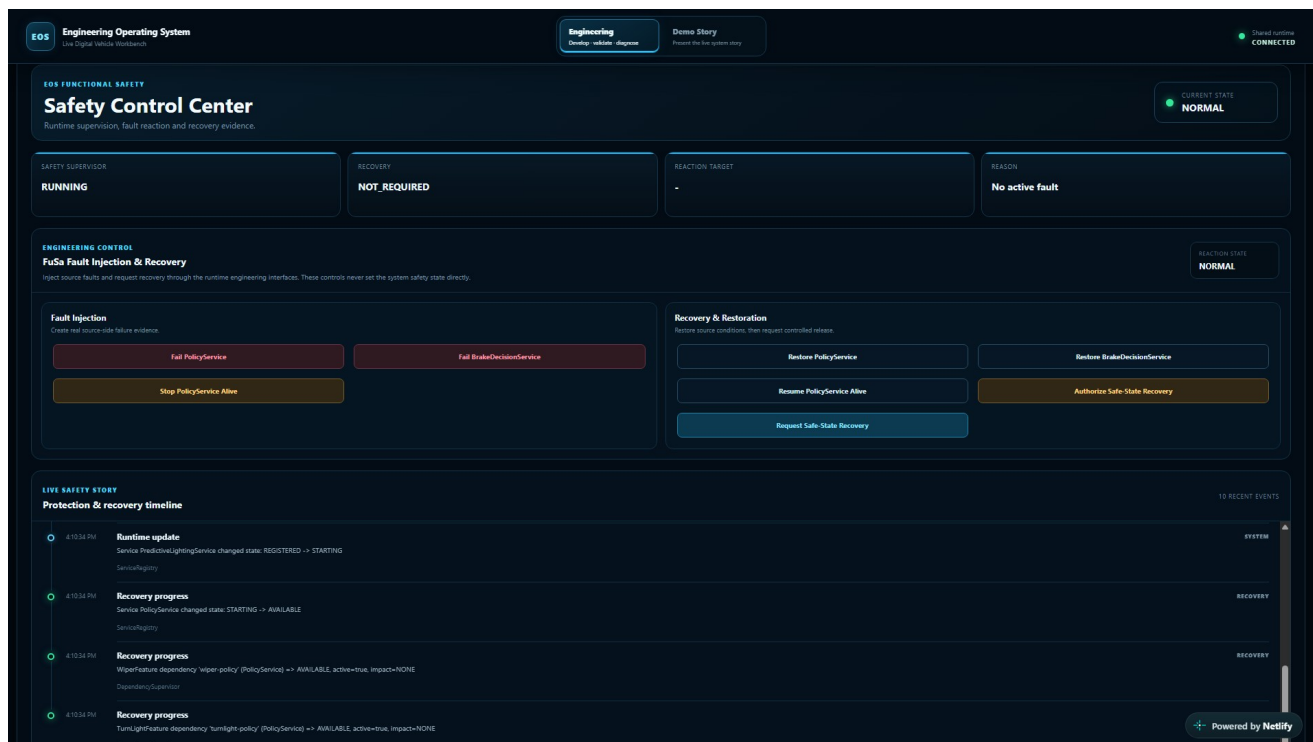


Figure 4-11 - Safety Control Center in NORMAL state

Caution: These controls are intended for engineering exploration. Later sections provide step-by-step safety workflows; first-time users should prefer those guided workflows rather than injecting arbitrary fault combinations.

4.13 Availability View

The **Availability** view deliberately separates several runtime concepts.

It contains four tables:

Nodes

Shows:

- Node
- State

Runtime Services

Shows:

- Service
- Host Node
- State

Deployed Runtime Components

Shows:

- Component
- Host Node
- Host State

Runtime Features

Shows:

- Feature
- Capability
- Operational state

This layout reflects the V1.0 ownership model:

```
Node state
  ≠
Service lifecycle
  ≠
Hosted-component reachability
  ≠
Feature capability/operation
```

For example, an online host node does not automatically prove that every component deployed to that host is functionally available.

The Availability view is therefore particularly useful during fault/recovery experiments.

4.14 Scenario View

The **Scenario** view is the detailed executable-validation view.

It lists all registered scenarios with:

- scenario name;
- description;
- tags;
- **Run** action.

After or during execution it provides:

- current status:
- **RUNNING**
- **IDLE**
- result:
- **PASS**
- **FAIL**
- **CANCELLED**
- execution duration;
- error information where applicable.

The **Execution Timeline** shows each scenario step with:

- step number;
- description;
- category;
- status;
- duration;
- timestamp.

Typical step states include:

- PASSED
- FAILED
- RUNNING

This is the primary view for validating *how* a scenario reached its final result rather than simply seeing PASS or FAIL.

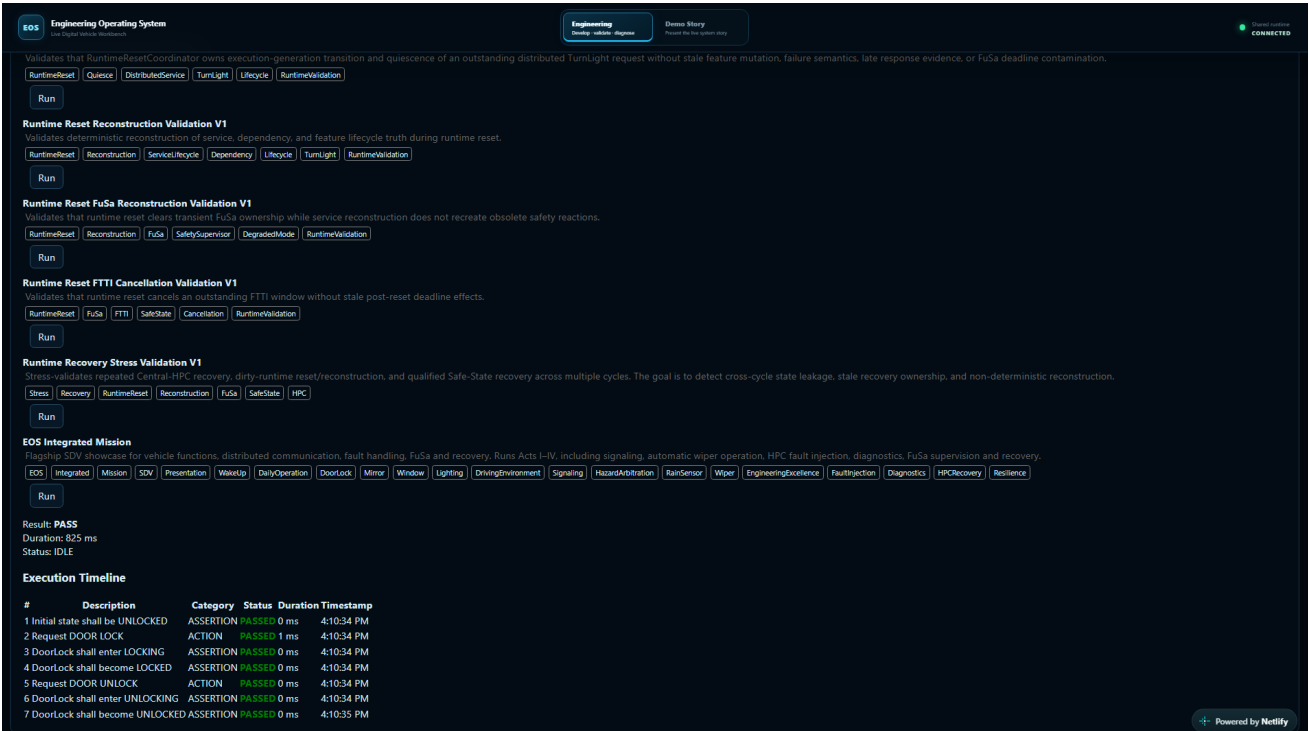


Figure 4-13 - Scenario Validation with PASS result

4.15 Demo Story Workspace

The **Demo Story** workspace presents selected system behavior as a guided live story.

It uses the same shared runtime as Engineering, but emphasizes:

- presentation;
- topology;
- communication flow;
- system state;
- evidence;
- end-to-end behavior.

V1.0 provides two Demo Stories:

- **Left Indicator Execution**
- **Central HPC Fault & Recovery**

The operator can select a story and use:

- **Run Live Story**
- **Stop Story**

where applicable.

The Demo Story includes live system evidence and presentation-oriented views of:

- vehicle/topology;
- protocol/communication activity;
- nodes;
- errors;
- runtime evidence;
- safety/recovery information;
- end-to-end execution stages.

The purpose is different from the Engineering workspace:

Engineering
→ inspect and diagnose the system in detail
Demo Story
→ communicate the live system behavior clearly

Because both workspaces share the same runtime, Demo Story remains connected to real runtime evidence rather than presenting a disconnected animation.

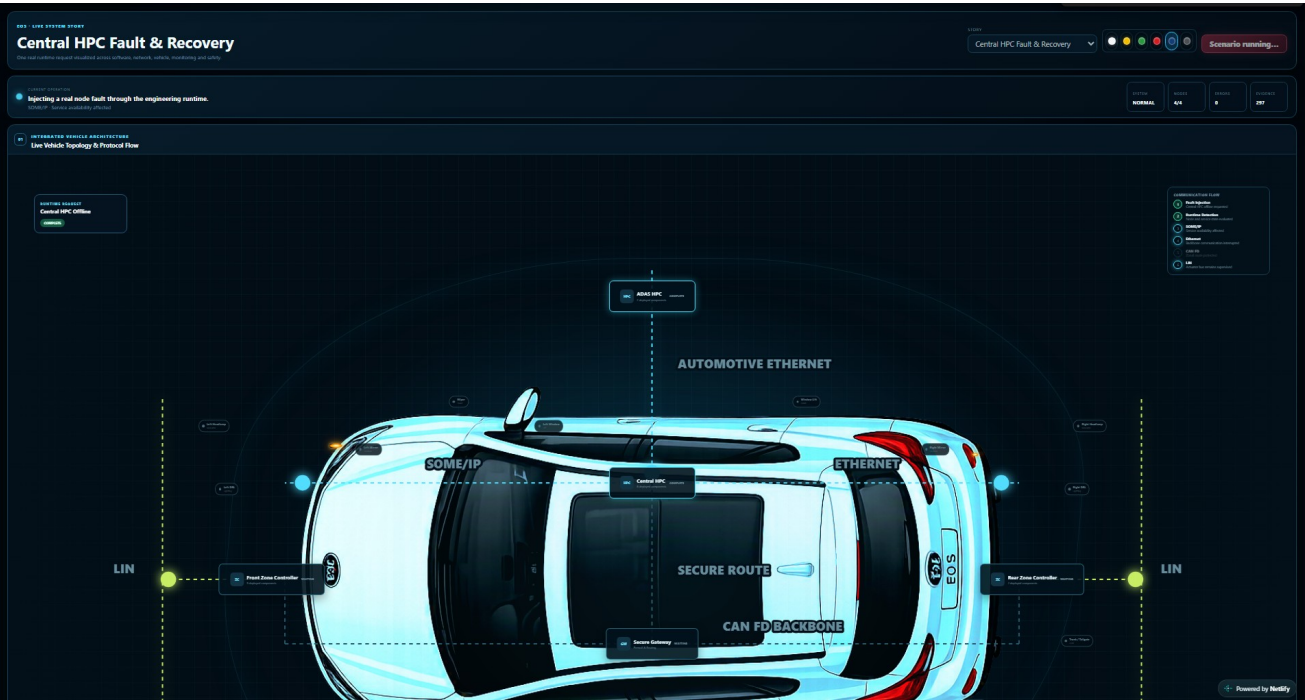


Figure 4-14 - Demo Story workspace

4.16 Choosing the Right View

A useful rule is to start with the question you want to answer.

User question	Recommended view
What is the overall system doing?	Runtime
Does vehicle behavior match software state?	Twin View
Where is software deployed and how does communication flow?	Topology
What diagnostic evidence has been recorded?	Diagnostics
What does a feature/component depend on?	Dependencies
What happened and in what order?	Timeline
Is a supervised component alive?	Alive
What is the Functional Safety state and reaction?	Safety
What is currently available?	Availability
Did the validation scenario pass, and which step failed?	Scenario
How can I present the system behavior end-to-end?	Demo Story

Users will often combine several views during one investigation.

For example:

```
Runtime
  ↓ notice DEGRADED state

Dependencies
  ↓ identify affected provider/consumer

Availability
  ↓ inspect current service/feature state

Diagnostics
  ↓ inspect fault evidence

Timeline
  ↓ reconstruct sequence

Safety
  ↓ inspect system-level reaction
```

That cross-view investigation is an intended way to use BCM Emulator V1.0.

5. Vehicle Configuration

BCM Emulator V1.0 allows the vehicle presentation to be changed without changing the underlying BCM engineering architecture.

Vehicle configuration is primarily a **presentation and product-variant choice**. It lets the user experience the same body-control platform with different vehicle packages and colors while continuing to work with the same shared runtime.

5.1 Vehicle Package Selection

The **Live Digital Vehicle** area provides a vehicle-package selector. The V1.0 Release Candidate contains five vehicle packages:

- **Hatchback**
- **Sedan**
- **SUV**
- **Sports**
- **Truck**

Selecting a package changes the vehicle representation used by the Live Digital Vehicle. The package selection demonstrates that the BCM platform is not tied to one visual vehicle body.

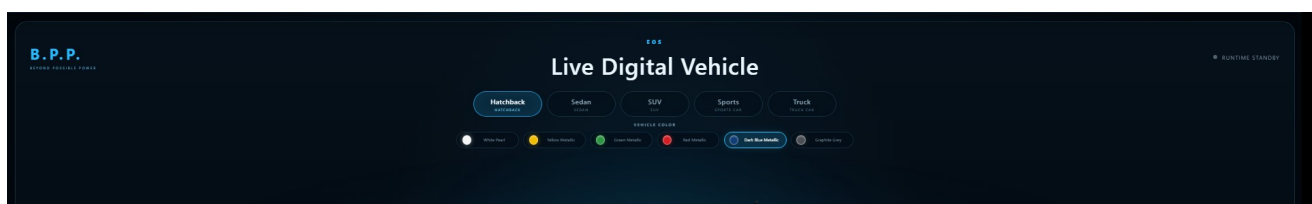


Figure 5-1 - Vehicle package and color selection

5.2 Vehicle Color Selection

The Live Digital Vehicle also provides vehicle-color selection. The V1.0 Release Candidate vehicle presentation supports six selectable colors:

- **White Pearl**
- **Yellow Metallic**
- **Green Metallic**
- **Red Metallic**
- **Dark Blue Metallic**
- **Graphite Grey**

Changing the color updates the vehicle appearance while leaving the engineering runtime unchanged. Color selection does not restart the runtime, reset a feature, alter dependencies or deployment, or change service lifecycle or Functional Safety state.

5.3 What Changes When the Vehicle Package Changes

A vehicle-package change primarily affects the visual representation of the vehicle.

Different packages can require package-specific visual geometry and positioning so that feature overlays remain aligned with the selected body shape. Examples include indicator lamps, interaction points, animation anchors and visual overlays.

```

Select another vehicle package
↓
Vehicle representation changes
↓
Visual anchors / overlays adapt
↓
BCM runtime continues
  
```

The package selector changes the vehicle presentation; it does not boot a separate BCM system.

5.4 What Remains Common

The core V1.0 engineering architecture remains common across the vehicle packages: BCM features, feature lifecycle, capability and operational states, services, dependencies, node/deployment model, communication, diagnostics, fault handling, Functional Safety, reset/reconstruction, scenario execution and engineering observability.

The Engineering views remain the same: Runtime, Twin View, Topology, Diagnostics, Dependencies, Timeline, Alive, Safety, Availability and Scenario.

Vehicle package describes how the vehicle is presented; it does not define a different runtime architecture.

5.5 Shared Runtime During Configuration Changes

Vehicle configuration remains part of the same running workbench. Changing the package or color does not create a second runtime.

Different presentation

≠

Different runtime

A user may, for example, set the vehicle to **IGNITION**, activate Hazard, select another vehicle package, and continue observing the same BCM runtime through the Engineering views.

5.6 Vehicle Configuration and Feature Behavior

The six V1.0 BCM feature domains remain available independently of the selected vehicle package:

Feature	Common across V1.0 vehicle packages
Door Lock	Yes
Fuel Flap	Yes
Window	Yes
Mirror	Yes
Turn Light / Hazard	Yes
Wiper	Yes

The selected package can change how a feature is **visualized**, but it does not create a package-specific implementation of the feature runtime. For example, signaling lamps may appear at different positions on different vehicle bodies while the underlying Turn Light / Hazard feature continues to use the same BCM runtime behavior.

5.7 Recommended User Experiment

1. Open **Engineering** → **Runtime** and set the vehicle to **IGNITION**.
2. Activate **Hazard**.
3. Switch between several vehicle packages, for example **Hatchback** → **SUV** → **Sports** → **Truck**.
4. Observe how the vehicle body and signaling visualization adapt.
5. Change the vehicle color.
6. Open **Runtime**, **Twin View**, or **Timeline** and confirm that the configuration change has not created a separate engineering runtime.

5.8 Configuration vs Runtime State

Configuration / presentation	Runtime / engineering state
Vehicle package	Vehicle power state
Vehicle color	Feature domain state
Package-specific visual anchors	Feature capability state
Vehicle body visualization	Operational state
Visual placement	Service lifecycle
	Dependency health
	Node state
	Safety state

The left side controls **how the system is presented**. The right side describes **what the running system is doing**.

6. BCM Features

BCM Emulator V1.0 contains six integrated body-control feature domains:

- Door Lock
- Fuel Flap
- Window
- Mirror
- Turn Light / Hazard
- Wiper

Each feature has its own domain behavior, but all six operate inside the same shared runtime architecture.

This means that a feature is not represented only by a visible vehicle animation. Depending on the feature, the user may also observe:

- domain state;
- capability state;
- operational state;
- dependencies;
- service/runtime behavior;
- diagnostics;
- communication;
- topology;
- safety impact;
- scenario evidence.

The following subsections introduce each feature from a user perspective.

6.1 Door Lock

Purpose

The Door Lock feature models vehicle door locking and unlocking as a discrete actuator function.

Its basic user-visible behavior is:

```

UNLOCKED
  ↓ Lock
LOCKING
  ↓
LOCKED
  ↓ Unlock
UNLOCKING
  ↓
UNLOCKED
  
```

The intermediate states are important because the emulator models a transition rather than jumping directly from one stable state to another.

User controls

The Live Digital Vehicle provides a context-dependent Door Lock control:

- **Lock Doors**
- **Unlock Doors**

The Runtime view also provides quick commands for:

- **Lock doors**
- **Unlock doors**

The displayed command depends on the current door-lock state.

Normal behavior

When the user requests locking:

1. the Door Lock feature accepts the request;
2. the actuator/domain state enters a locking transition;
3. the feature is operationally active while the transition is executing;
4. the domain state reaches **LOCKED**;
5. operational state returns to **INACTIVE**.

Unlocking follows the reverse transition.

A typical stable state can therefore be:

```
Domain      = LOCKED
Capability  = AVAILABLE
Operational = INACTIVE
```

This means the doors are locked, the feature remains usable, and no lock/unlock movement is currently executing.

What to observe

Useful views include:

- **Live Digital Vehicle** — visible locked/unlocked state;
- **Runtime** — current runtime picture;
- **Twin View** — physical vs software synchronization;
- **Timeline** — request and state-transition events;
- **Availability** — Door Lock capability and operational state;
- **Scenario** — detailed validation steps.

Related validation

The recommended introductory scenario is:

DoorLock Discrete Actuator Scenario V1

This scenario is also used in Section 2 as the first executable validation example.

6.2 Fuel Flap

Purpose

The Fuel Flap feature models opening and closing the vehicle fuel flap as a discrete actuator function.

Like Door Lock, it follows a command-to-transition-to-stable-state model.

Conceptually:

```
CLOSED
  ↓ Open
OPENING
  ↓
OPEN
  ↓ Close
CLOSING
  ↓
CLOSED
```

User interaction

Fuel Flap is part of the integrated BCM feature set and can be exercised through the corresponding feature/scenario interaction provided by the V1.0 platform.

The exact visible control location may differ from the more prominent Live Digital Vehicle controls documented earlier, so users should use the feature/scenario interface available in the Public Preview rather than assuming a separate global quick-command button.

Normal behavior

A normal open operation consists of:

1. request;
2. transition into the opening state;
3. actuator movement;
4. stable **OPEN** state;
5. operational state returning to **INACTIVE**.

Closing follows the corresponding reverse sequence.

What to observe

Useful views include:

- **Twin View** — synchronized domain/runtime representation;
- **Timeline** — command and transition sequence;
- **Availability** — feature capability and operational state;
- **Scenario** — feature validation steps and PASS/FAIL evidence.

Fuel Flap is particularly useful for understanding the difference between:

```

stable domain state
    and
currently active operation

```

A flap can be **OPEN** while the feature is operationally **INACTIVE**.

6.3 Window

Purpose

The Window feature models a continuous actuator.

Unlike Door Lock or Fuel Flap, the Window feature is not only concerned with two discrete stable states. It also represents motion and position.

Typical user-visible behavior includes:

- opening;
- closing;
- stopping during motion;
- maintaining the resulting position.

User controls

The Live Digital Vehicle uses context-dependent controls such as:

- **Open Windows**
- **Close Windows**
- **Stop Windows**

The available command changes according to the current Window state.

Normal behavior

A simplified Window sequence is:

```

request OPEN
  ↓
window starts moving
  ↓
Operational = ACTIVE
  ↓
position changes over time
  ↓
target reached or STOP requested
  ↓
movement stops
  ↓
Operational = INACTIVE

```

The final Window position remains part of the domain state even after movement stops.

Continuous behavior

Window demonstrates an important difference from discrete features:

```

Discrete actuator
→ state transition between defined stable states

Continuous actuator
→ movement over time toward a position

```

For this reason, the user may observe intermediate position values rather than only an OPEN/CLOSED result.

What to observe

Useful views include:

- **Live Digital Vehicle** — visible Window movement;
- **Twin View** — physical movement vs runtime representation;
- **Timeline** — start/stop and state events;
- **Availability** — capability and operational state;
- **Scenario** — validation of continuous actuator behavior.

6.4 Mirror

Purpose

The Mirror feature is another continuous actuator example.

It models mirror adjustment over time rather than a single instantaneous state change.

User controls

The Live Digital Vehicle provides:

- **Move Mirrors Left**
- **Move Mirrors Right**
- **Stop Mirrors**

Normal behavior

A typical adjustment flow is:

```

Move Left / Move Right requested
  ↓
Mirror actuator starts movement
  ↓
Operational = ACTIVE
  ↓
position changes
  ↓
Stop or target reached
  ↓
Operational = INACTIVE
  
```

The resulting mirror position is retained as domain state.

Relationship to Window

Window and Mirror share the same broad continuous-actuator behavior pattern:

- movement over time;
- position state;
- start/stop control;
- active vs inactive operation.

However, each remains an independent vehicle feature with its own domain state and controls.

What to observe

Useful views include:

- **Live Digital Vehicle** — command and visual response;
- **Twin View** — synchronized physical/runtime state;
- **Timeline** — movement commands and transitions;
- **Availability** — feature capability and operational state;
- **Scenario** — validation evidence.

6.5 Turn Light / Hazard

Purpose

The Turn Light feature models vehicle signaling behavior including:

- Left Turn;
- Right Turn;
- Hazard.

This feature is more than a simple on/off lamp switch. It includes signaling behavior, arbitration and coordinated lamp output.

User controls

The Live Digital Vehicle provides:

- **Left Turn**
- **Right Turn**
- **Hazard**

The Runtime view also provides the corresponding quick commands.

6.6 Wiper

Purpose

The Wiper feature models periodic actuator motion and operating modes.

V1.0 provides the user modes:

- **OFF**
- **AUTO**
- **SINGLE**
- **LOW**

This feature is especially useful because it combines:

- periodic motion;
- operating mode;
- sensor-dependent behavior;
- conditional dependency handling.

OFF

In **OFF**, the Wiper is not performing regular wiping.

The mechanism returns to or remains in its parked/inactive condition.

SINGLE

SINGLE requests one wipe cycle.

Conceptually:

```

SINGLE requested
↓
wipe cycle starts
↓
Operational = ACTIVE
↓
periodic actuator movement
↓
cycle completes
↓
Wiper parks
↓
Operational = INACTIVE

```

LOW

LOW requests continuous low-speed wiping.

The Wiper remains operationally active while the repeated wiping behavior continues.

AUTO

AUTO introduces a conditional dependency on Rain Sensor information.

When AUTO is selected, the Rain Sensor becomes relevant to Wiper operation.

Conceptually:

```

AUTO selected
↓
Rain Sensor dependency active
↓
sensor evidence evaluated
↓
automatic wiping behavior

```

When AUTO is not selected, the Rain Sensor dependency can be inactive.

This is a useful example of a runtime dependency whose relevance depends on the current operating mode.

Rain Sensor interaction

The Runtime view provides:

- **Rain sensor**
- **Clear rain**

These controls allow the user to provide or clear rain-related input evidence.

A useful experiment is:

1. set vehicle state to **IGNITION**;
2. select Wiper **AUTO**;
3. activate **Rain sensor**;
4. observe automatic Wiper behavior;
5. select **Clear rain**;
6. observe the resulting Wiper response;
7. inspect **Dependencies** to see the Rain Sensor relationship.

What to observe

Useful views include:

- **Live Digital Vehicle** — Wiper mode and movement;
- **Runtime** — Rain Sensor commands;
- **Twin View** — physical/runtime Wiper state;
- **Dependencies** — conditional Rain Sensor dependency;
- **Timeline** — mode/input/runtime events;
- **Availability** — Wiper capability and operational state;
- **Scenario** — periodic-motion validation.

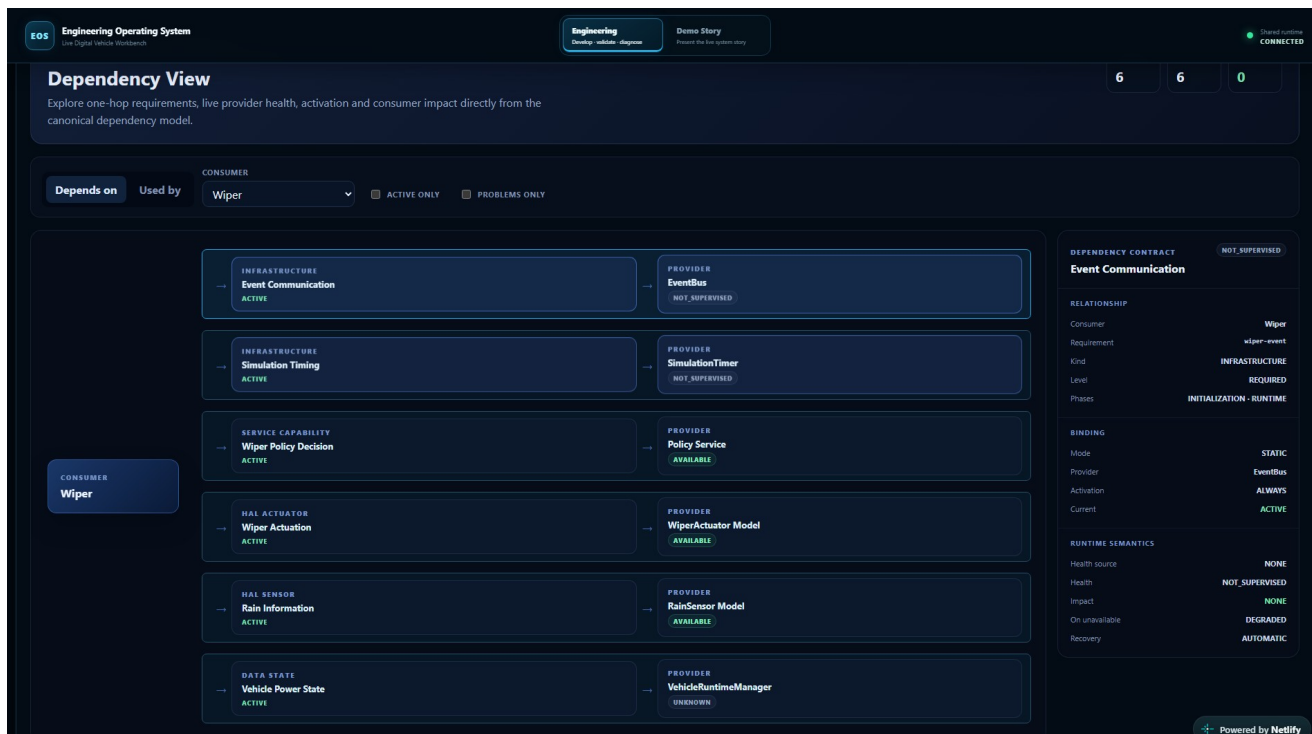


Figure 6-6 - Dependency relationships used for engineering investigation

6.7 Comparing the Six Features

The feature portfolio intentionally covers different body-control behavior types.

Feature	Main behavior type	Typical interaction
Door Lock	Discrete actuator	Lock / Unlock
Fuel Flap	Discrete actuator	Open / Close
Window	Continuous actuator	Open / Close / Stop
Mirror	Continuous actuator	Move Left / Move Right / Stop
Turn Light / Hazard	Signaling / arbitration	Left / Right / Hazard
Wiper	Periodic motion / mode-driven	OFF / AUTO / SINGLE / LOW

This variety is important because BCM Emulator is intended to demonstrate reusable engineering concepts across different kinds of vehicle behavior rather than one implementation repeated six times.

6.8 Common Runtime Concepts Across Features

Although the domain behavior differs, all features participate in the common runtime model.

For every feature, users should distinguish:

Domain state

What the vehicle function is physically/logically doing.

Examples:

```
Door Lock → LOCKED
Window    → position / movement state
Wiper     → PARKED / wiping mode
```

Capability state

Whether the feature can currently provide its intended function:

- INITIALIZED
- AVAILABLE
- DEGRADED
- UNAVAILABLE
- FAILED

Operational state

Whether the feature is currently executing its operation:

- ACTIVE
- INACTIVE

This creates a reusable interpretation model:

```
Feature
├── Domain State
│   └── What is the vehicle function doing?
├── Capability State
│   └── Can the function currently be provided?
└── Operational State
    └── Is the function executing right now?
```

The same distinction applies whether the feature is discrete, continuous, signaling-based or periodic.

6.9 A Recommended Feature Exploration Workflow

For any BCM feature, a useful first exploration sequence is:

```
1. Power vehicle to IGNITION
   ↓
2. Operate one feature
   ↓
3. Observe Live Digital Vehicle
   ↓
4. Check Twin View
   ↓
5. Check Availability
   ↓
6. Inspect Timeline
   ↓
7. Inspect Dependencies
   ↓
8. Run the corresponding feature scenario
```

This workflow helps connect visible behavior to runtime state and validation evidence.

The next section goes deeper into the state model used across all features and the wider BCM runtime.

7. Runtime and System States

BCM Emulator V1.0 exposes several state models at the same time. They describe different concerns and are intentionally owned by different runtime mechanisms.

This section explains how to read them without mixing their meanings.

7.1 Vehicle Power State

The vehicle runtime uses three power states:

UI label	Runtime state	Meaning
OFF	OFF	Vehicle runtime is not in an active operating mode
ACCESSORY	ACCESSORY	Accessory-level operation is permitted
IGNITION	IGNITION_ON	Normal powered vehicle operation

The Runtime view exposes these states through:

Runtime control → **Vehicle state**

Vehicle power is a domain/runtime permission condition for several features.

For example:

- Window movement requires **ACCESSORY** or **IGNITION_ON**;
- Mirror movement requires **ACCESSORY** or **IGNITION_ON**;
- Turn Light operation is permitted in **ACCESSORY** or **IGNITION_ON**;
- Wiper operation requires **IGNITION_ON**.

Door Lock and Fuel Flap use their own feature semantics and do not use the same power-gating rule.

Important: Vehicle power **OFF** is not a complete runtime reset.

7.2 Feature Capability State

Every registered BCM feature has a capability state.

V1.0 uses:

State	User interpretation
INITIALIZED	The feature exists in the runtime, but usable capability has not yet been established
AVAILABLE	The feature can provide its intended capability
DEGRADED	The feature remains usable with reduced capability
UNAVAILABLE	The intended capability cannot currently be provided
FAILED	An internal feature failure owns the highest-severity feature state

Capability is derived from lifecycle evidence and active reasons.

The V1.0 precedence is:

```

FAILED
↓
UNAVAILABLE
↓
DEGRADED
↓
AVAILABLE
↓
INITIALIZED

```

A more severe active lifecycle reason therefore dominates a less severe one.

For example, if a feature has both a degraded dependency and an internal failure, its resulting capability is **FAILED**.

7.3 Feature Operational State

Feature operational state answers a different question:

Is the feature currently performing its operation?

V1.0 uses:

- **ACTIVE**
- **INACTIVE**

A feature may therefore be:

```
Capability = AVAILABLE
Operational = INACTIVE
```

when it is healthy but not currently executing.

A feature in **DEGRADED** capability may still be allowed to remain operational. A feature in **UNAVAILABLE** or **FAILED** capability is forced to **INACTIVE**.

7.4 Domain State

Domain state is owned by each vehicle feature and describes the feature-specific behavior.

Examples include:

```
Door Lock
UNLOCKED → LOCKING → LOCKED → UNLOCKING → UNLOCKED

Fuel Flap
CLOSED → OPENING → OPEN → CLOSING → CLOSED
```

Window, Mirror, Turn Light and Wiper use domain models appropriate to continuous motion, signaling or periodic motion.

Domain state should be interpreted together with capability and operational state, but it is not replaced by either one.

7.5 Service Lifecycle State

Lifecycle-managed runtime services use:

- **REGISTERED**
- **STARTING**
- **AVAILABLE**
- **DEGRADED**
- **FAILED**
- **RECOVERING**
- **UNAVAILABLE**

Registration and operational availability are separate concepts.

V1.0 explicitly lifecycle-manages:

- **PolicyService**
- **PredictiveLightingService**
- **BrakeDecisionService**

Other registered objects can appear in deployment or runtime structures without becoming lifecycle-managed services.

For this reason:

```
registered object
≠
lifecycle-managed service
```

and:

```
host node online
≠
service automatically available
```

7.6 Node State

The virtual vehicle topology models computing nodes as:

- **ONLINE**
- **OFFLINE**

V1.0 contains four principal topology nodes:

- Central HPC
- ADAS HPC
- Front Zone Controller
- Rear Zone Controller

Node state can affect components deployed to the node, but the resulting effect depends on which runtime mechanism owns each component.

A node becoming **OFFLINE** is therefore source evidence, not a universal command to set every hosted object to the same state.

7.7 Dependency Health and Impact

Dependency state is represented separately from feature and service state.

Dependency health can be:

- `AVAILABLE`
- `DEGRADED`
- `UNAVAILABLE`
- `UNKNOWN`

Consumer impact can be:

- `NONE`
- `DEGRADED`
- `UNAVAILABLE`

These are intentionally different dimensions.

Conceptually:

```

Provider condition
  ↓
Dependency health
  ↓
Dependency contract
  ↓
Consumer impact
  ↓
Feature lifecycle reason
  ↓
Feature capability

```

The dependency model also records whether the dependency is currently **ACTIVE** or **INACTIVE**.

This matters for conditional dependencies such as the Wiper AUTO relationship to `PolicyService` and `RainSensorModel`.

7.8 Functional Safety State

The V1.0 Release Candidate uses one canonical public safety-state vocabulary:

- `NORMAL`
- `DEGRADED`
- `SAFE STATE`

The main Safety Control Center **Current state** is a high-level aggregate derived from current runtime evidence:

```

Safe State active
  → SAFE STATE

otherwise:
  degraded-mode active
  or one/more nodes offline
  or active ERROR diagnostics exist
    → DEGRADED

otherwise
  → NORMAL

```

The **Engineering Control** reaction indicator uses the same three labels, but its scope is narrower: it reflects the active safety-reaction state itself.

This means that identical labels do not imply identical state ownership. For example, the high-level Current state can become `DEGRADED` because of an offline node or active ERROR diagnostic even when no separate degraded-mode safety reaction has been selected.

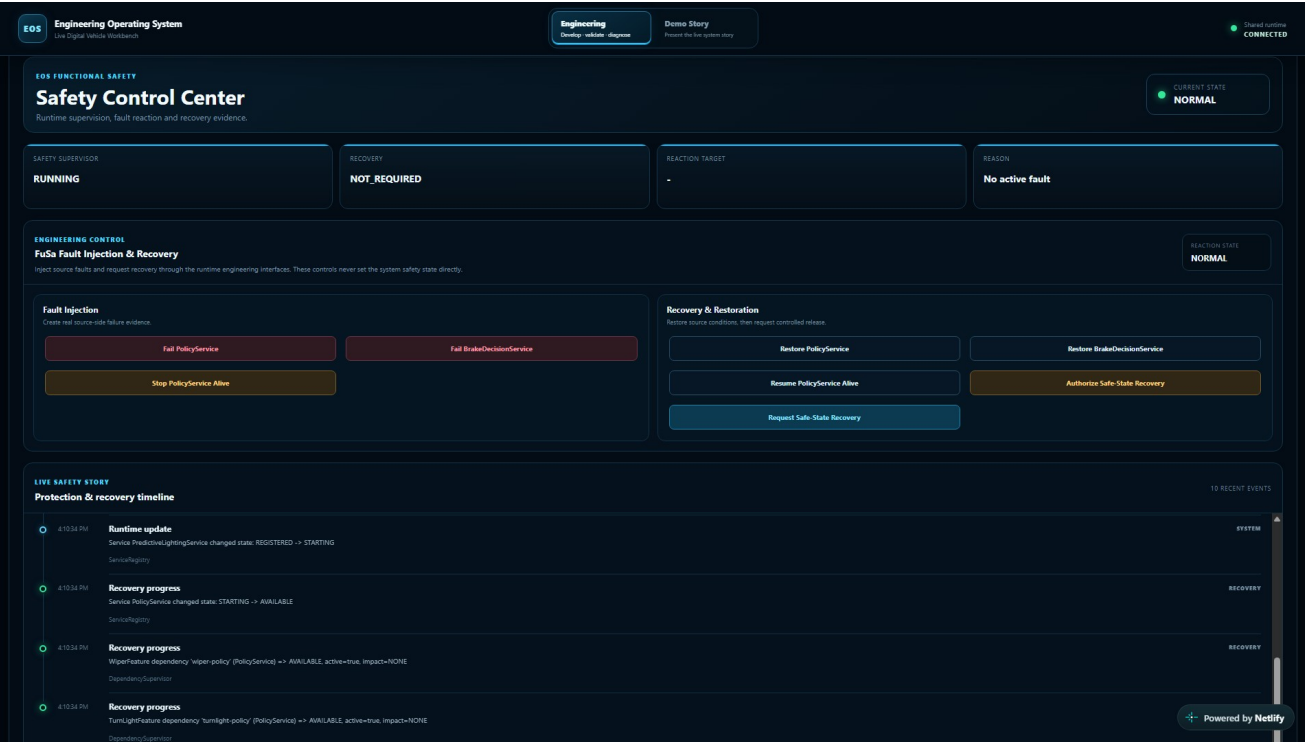


Figure 7-1 - Safety Control Center showing NORMAL system state

7.9 Runtime Reset State

The reset coordinator has its own state:

- **READY**
- **RESETTING**
- **FAILED**

and its own phases:

- **IDLE**
- **QUIESCE**
- **RESTORE_SOURCE**
- **RESTORE_FEATURE_RUNTIME**
- **RECONSTRUCT**
- **REBUILD_EVIDENCE**
- **SETTLE**

These states belong to runtime reconstruction and should not be confused with vehicle power, feature capability or system safety state.

The complete reset lifecycle is described in Section 16.

7.10 State Ownership Summary

A useful V1.0 ownership map is:

Question	State owner / model
Is vehicle power on?	Vehicle Runtime
What is a feature doing?	Feature domain state
Can the feature provide its function?	Feature lifecycle/capability
Is the feature operating right now?	Feature operational state
Is a runtime service available?	Service lifecycle
Is a compute node online?	Node/topology state
Is a dependency healthy?	Dependency model
Is the system protected/degraded?	Functional Safety
Is reconstruction in progress?	Runtime Reset Coordinator

This separation is one of the central design principles of the emulator.

8. Fault Injection and Recovery

Fault injection is a first-class engineering capability in BCM Emulator V1.0. The platform does not only show successful operation. It allows the user to introduce controlled faults and observe their consequences across communication, services, dependencies, features, diagnostics and Functional Safety.

8.1 Two Main Fault-Injection Areas

V1.0 exposes fault controls in two main areas:

Topology — Engineering Fault Injection

The Topology view provides controls for infrastructure/network experiments.

Safety — FuSa Fault Injection & Recovery

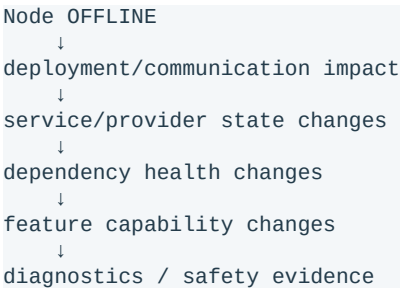
The Safety view provides service/liveness faults and controlled safety-recovery actions. These controls operate through runtime engineering interfaces and create source-side evidence. They do not directly paint a desired state into the UI.

8.2 Topology Fault Controls

The **Engineering Fault Injection** console provides:

Control	V1.0 action
Front ZC Offline	Marks the Front Zone Controller offline
Central HPC Offline	Marks the Central HPC offline
Ethernet Latency	Sets inter-node latency to 500 ms
Packet Loss	Sets packet-loss probability to 0.5
Recover All	Restores offline nodes, clears network faults, returns packet loss to 0 and inter-node latency to 20 ms

A node-offline action can propagate through several architectural layers:



The exact chain depends on the affected component and its contracts.

8.3 Service Fault Controls

The Safety view provides:

Fault Injection

- **Fail PolicyService**
- **Fail BrakeDecisionService**
- **Stop PolicyService Alive**

These controls exercise different kinds of evidence:

- explicit service lifecycle failure;
- ASIL-D safety-related service failure;
- loss of alive/heartbeat evidence.

The resulting reaction is determined by the runtime safety mechanisms.

8.4 Recovery and Restoration Controls

1 SAFE STATE active

2 Safety evidence

3 Recovery diagnostics

4 Event evidence

Safety recovery sequence - fault evidence, recovery evidence and restoration

The Safety view provides:

- **Restore PolicyService**
- **Restore BrakeDecisionService**
- **Resume PolicyService Alive**
- **Authorize Safe-State Recovery**
- **Request Safe-State Recovery**

These controls deliberately separate three concepts.

Technical restoration

Examples:

- a service transitions back to **AVAILABLE**;
- heartbeat evidence resumes;
- an offline node comes online.

Recovery qualification / authorization

For Safe State, V1.0 requires an explicit authorization gate:

Authorize Safe-State Recovery

Recovery request

The system must then receive:

Request Safe-State Recovery

The resulting model is:

```

fault occurs
↓
SAFE STATE entered
↓
technical source recovers
↓
SAFE STATE remains latched
↓
recovery is authorized
↓
recovery is requested
  
```

```

↓
final recovery conditions checked
↓
SAFE STATE exited

```

Technical recovery alone is therefore not equivalent to safety release.

8.5 Why Recovery Can Be Denied

The Safe-State implementation rejects recovery when required conditions are not satisfied.

Two explicit V1.0 denial conditions are demonstrated by the Safe State Recovery scenario:

- escalation is still active;
- the initiating fault has not recovered.

A denied request produces runtime evidence and a diagnostic entry.

This behavior is intentional because the recovery request is a decision point, not a command to force the system into a healthy display state.

8.6 Automatic Degraded-Mode Recovery

V1.0 also contains automatic recovery logic for registered services that own a system-level `DEGRADED_MODE` reaction.

The recovery coordinator requires evidence including:

```

service AVAILABLE
+
fresh ComponentAlive evidence
↓
reaction-release decision

```

Heartbeat evidence from before the recovery is not reused as proof.

This is different from Safe-State release, which remains explicitly controlled.

8.7 Recommended Fault-Investigation Flow

When injecting a fault, use a deliberate observation sequence:

```

1. Confirm baseline
↓
2. Inject one fault
↓
3. Observe Topology / Availability
↓
4. Inspect Dependencies
↓
5. Inspect Diagnostics
↓
6. Inspect Safety
↓
7. Inspect Timeline
↓
8. Restore the source condition
↓
9. Verify current-state recovery

```

For Safe-State experiments, add:

```

10. Authorize Safe-State Recovery
11. Request Safe-State Recovery
12. Verify Safe-State exit

```

Keeping fault injection one change at a time makes the causal chain easier to understand.

8.8 Fault Injection Is Not Runtime Reset

`Recover All` restores the network-fault controls managed by the Topology engineering interface.

It is **not** the canonical full-runtime reset/reconstruction mechanism.

Similarly:

- putting a node online is not a full reset;
- restoring a service is not a full reset;
- setting vehicle power to OFF is not a full reset;
- clearing rain is not a full reset.

The complete baseline reconstruction is owned by `RuntimeResetCoordinator` and is automatically used at scenario boundaries.

9. Working with Scenarios

Executable scenarios turn architecture and runtime behavior into repeatable validation.

The V1.0 Release Candidate registers **24 scenarios**.

9.1 What a Scenario Contains

Every scenario provides:

- a name;
- a description;
- tags;
- executable steps;
- assertions;
- a final result.

Scenario steps use categories such as:

- `ACTION`
- `FAULT`
- `ASSERTION`
- `UNKNOWN`

and step status values:

- `PENDING`
- `RUNNING`
- `PASSED`
- `FAILED`

The Scenario view shows the execution timeline using this information.

9.2 Starting a Scenario

A scenario can be started from:

Engineering → **Runtime** → **Scenario validation**

or from:

Engineering → **Scenario**

The Runtime view provides a selector and **Run scenario** button.

The Scenario view lists all registered scenarios and provides an individual **Run** action.

Only one scenario is allowed to run at a time.

9.3 Automatic Baseline Reset

Before the selected scenario executes, `ScenarioRunner` calls the canonical runtime reset.

The sequence is:

```
scenario selected
  ↓
step/cancellation state cleared
  ↓
canonical runtime reset
  ↓
known runtime baseline reconstructed
  ↓
scenario steps execute
```

If the baseline reset fails, the scenario fails rather than continuing from an uncertain state.

This is why users normally do not need to prepare the runtime manually before starting a scenario.

9.4 What Happens After a Scenario

The runtime is intentionally **not** reset immediately after scenario completion.

The final state remains visible for inspection.

This allows the user to examine:

- Runtime;
- Twin View;
- Topology;

- Dependencies;
- Diagnostics;
- Timeline;
- Safety;
- Availability.

The next scenario owns establishment of the next clean baseline.

9.5 PASS, FAIL and CANCELLED

Scenario-level result values are:

- PASS
- FAIL
- CANCELLED

A successful scenario means its implemented assertions completed without failure.

It does not mean that every subsystem remained nominal throughout the scenario. Many scenarios intentionally create faults and pass only when the expected degraded, failed or recovery behavior is observed.

For example:

```
intentional fault
+
expected system reaction
+
correct recovery behavior
=
scenario PASS
```

9.6 General Stop Behavior in V1.0

The scenario runtime supports cancellation internally, but the general Engineering Scenario panel does not expose a universal **Stop scenario** button in the V1.0 Release Candidate UI.

The Demo Story workspace has story-specific stop behavior for its presentation flow.

For normal Scenario validation, users should allow the selected scenario to complete before starting another.

9.7 How to Read the Execution Timeline

For each step, inspect:

- **#** — step order;
- **Description** — what is being performed or validated;
- **Category** — action/fault/assertion;
- **Status** — running/passed/failed;
- **Duration** — step execution time;
- **Timestamp** — when execution began.

If a scenario fails, the failed step is the best first clue.

Then correlate that step with:

1. Timeline;
2. Diagnostics;
3. Availability;
4. Dependencies;
5. Safety where relevant.

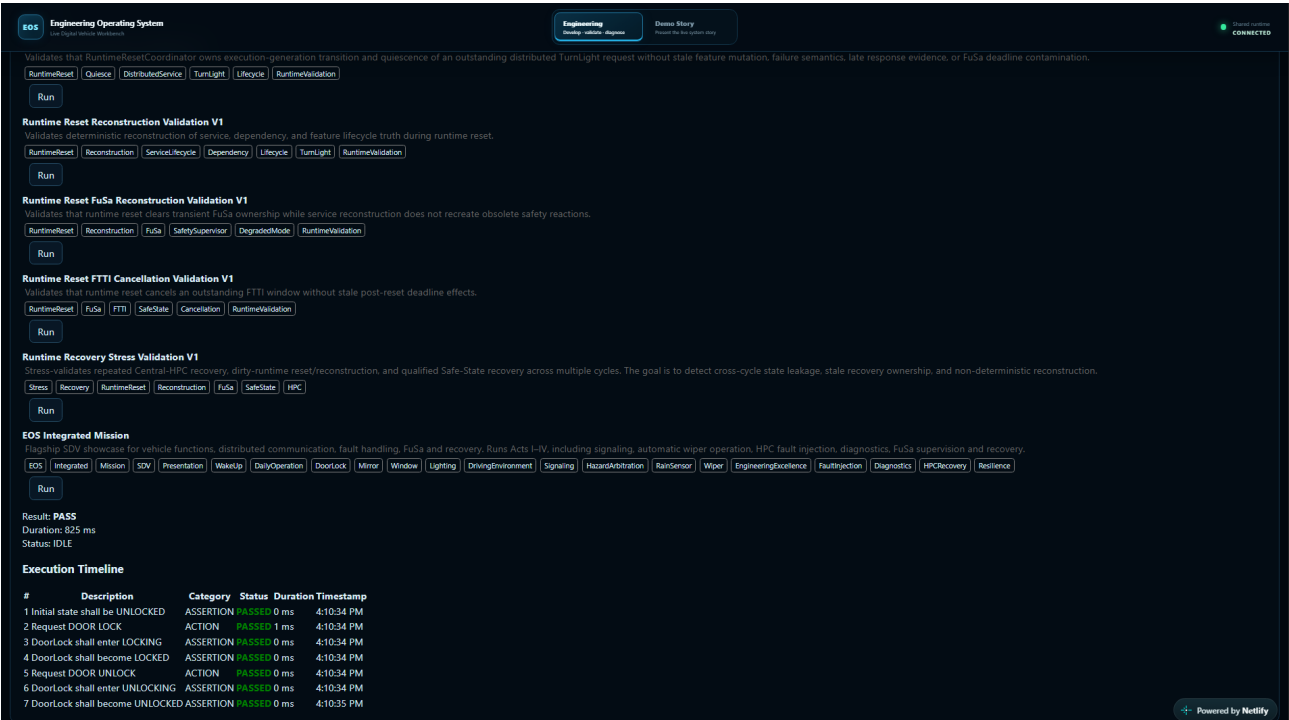
9.8 Guide Grouping of the 24 V1.0 Scenarios

For this User Guide, the scenarios are organized into four user-facing groups:

Group	Count	Covered in
Feature behavior / feature robustness	11	Section 10
Runtime, reset, cancellation and recovery	8	Section 11
Functional Safety	4	Section 12
Integrated end-to-end mission	1	Section 13

This grouping is for navigation in the guide; scenario tags in the runtime provide additional technical classification.

10. Feature Scenarios



Executable feature scenario with PASS result and step-by-step evidence

This section covers the scenarios whose primary purpose is validating a BCM feature or feature-pattern behavior. The general execution method is the same for every scenario:

Engineering → Runtime → Scenario validation → select scenario → Run scenario

OR:

Engineering → Scenario → Run

Each scenario receives a canonical reset before execution.

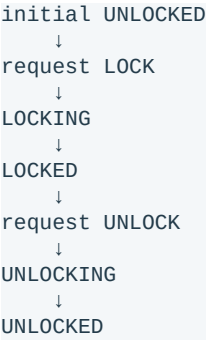
10.1 DoorLock Discrete Actuator Scenario V1

Purpose

Validate Door Lock command handling, discrete actuator runtime state and semantic state.

Validation sequence

The scenario verifies:



What to observe

- Live Digital Vehicle Door Lock state;
- Scenario Execution Timeline;
- Timeline business events;
- Availability feature state.

Expected result

PASS confirms that stable state, target state, transition state and semantic feature state agree throughout the lock/unlock cycle.

10.2 FuelFlap Discrete Actuator Scenario V1

Purpose

Validate Fuel Flap command handling and discrete actuator semantics.

Validation sequence

```
initial CLOSED
  ↓
request OPEN
  ↓
OPENING
  ↓
OPEN
  ↓
request CLOSE
  ↓
CLOSING
  ↓
CLOSED
```

Expected result

PASS confirms consistent transition and stable-state behavior.

10.3 Window Continuous Actuator Scenario V3

Purpose

Validate Window command handling, physical movement and semantic interpretation.

Key steps

- set **ACCESSORY**;
- verify initial **CLOSED**;
- request Window Down and verify opening;
- request Window Up and verify closing;
- request partial opening;
- issue **STOP**;
- verify stopped intermediate position.

Engineering meaning

This scenario demonstrates that a continuous actuator is not limited to two stable endpoints. An intermediate physical position is a valid state.

10.4 Mirror Continuous Actuator Scenario V2

Purpose

Validate Mirror motion, endpoint behavior and stopping at an intermediate position.

Key steps

- set **ACCESSORY**;
- verify initial state;
- move left;
- verify left endpoint;
- move right;
- verify right endpoint;
- move left again;
- stop;
- verify intermediate position.

Engineering meaning

The scenario validates the same reusable continuous-motion concepts used by Window while preserving Mirror-specific domain semantics.

10.5 TurnLight Signaling Pattern Scenario V1

Purpose

Validate signaling timing, priority resolution, semantic state and physical lamp outputs.

Key sequence

```

IGNITION_ON
↓
LEFT requested
↓
LEFT ON/OFF flashing phases
↓
HAZARD activated
↓
HAZARD wins all four channels
↓
HAZARD released
↓
LEFT request resumes
↓
LEFT deactivated
↓
RIGHT requested
↓
vehicle power OFF
↓
all TurnLight outputs cancelled

```

What to observe

This scenario is especially useful in:

- Live Digital Vehicle;
- Twin View;
- Timeline;
- Topology / communication evidence;
- Scenario timeline.

Expected result

PASS confirms signaling arbitration and physical outputs remain consistent with the effective signaling request.

10.6 TurnLight Cancellation Validation V1

Purpose

Validate lifecycle cancellation of an outstanding distributed Turn Light policy request.

What it protects against

The scenario verifies that quiescence does not incorrectly create:

- Turn Light rejection semantics;
- stale domain-state mutation;
- physical lamp actuation;
- late-response evidence;
- FuSa deadline contamination.

Engineering meaning

Cancellation caused by lifecycle quiescence is not the same as a business/runtime failure.

This is important during reset and reinitialization.

10.7 Wiper Periodic Motion Pattern Scenario V1

Purpose

Validate the main Periodic Motion behavior of Wiper.

Key sequence

- set `IGNITION_ON`;
- establish Wiper `OFF`;
- run one `SINGLE` wipe;
- verify outward motion;
- verify exactly one cycle and park;
- run continuous `LOW`;
- request `OFF` while moving;
- verify graceful stop and park;
- select `AUTO`;
- report rain;
- verify `AUTO` resolves to `LOW`;
- clear rain;
- verify graceful park.

Expected result

`PASS` validates command mapping, motion execution, single-cycle semantics, continuous operation, graceful stop and `AUTO` resolution.

10.8 Wiper Physical Motion Validation V1

Purpose

Validate the physical-motion model behind Wiper.

Assertions include

- normalized position without overshoot;
- monotonic outward motion;
- direction reversal at the outer limit;
- monotonic return toward park;
- exact park completion for a single cycle.

Engineering meaning

This scenario focuses below the user-facing Wiper mode and validates the physical movement representation itself.

10.9 Wiper Periodic Motion Fault Validation V1

Purpose

Validate Wiper fault detection and safe motion handling.

Faults exercised

- blocked actuator;
- unavailable actuator.

Expected behavior

For each fault, the scenario verifies that the Periodic Motion runtime enters a faulted condition and stops motion safely. It then explicitly resets/restores the Wiper runtime and actuator before continuing.

10.10 Wiper Periodic Motion Network Validation V1

Purpose

Validate the distributed service path used by Wiper `AUTO`.

Key sequence

```

Central HPC OFFLINE
↓
rain reported
↓
AUTO policy path unavailable
↓
Wiper fails safe to OFF
↓
Central HPC ONLINE
↓
AUTO re-evaluated
↓
LOW motion resumes
↓
rain stops
↓
Wiper parks

```

Engineering meaning

AUTO is intentionally different from direct manual Wiper modes because it uses conditional sensor and policy dependencies.

10.11 Wiper Cancellation Validation V1

Purpose

Validate cancellation of a Wiper AUTO policy request that belongs to an obsolete evaluation generation.

Expected protection

The cancelled request must not create:

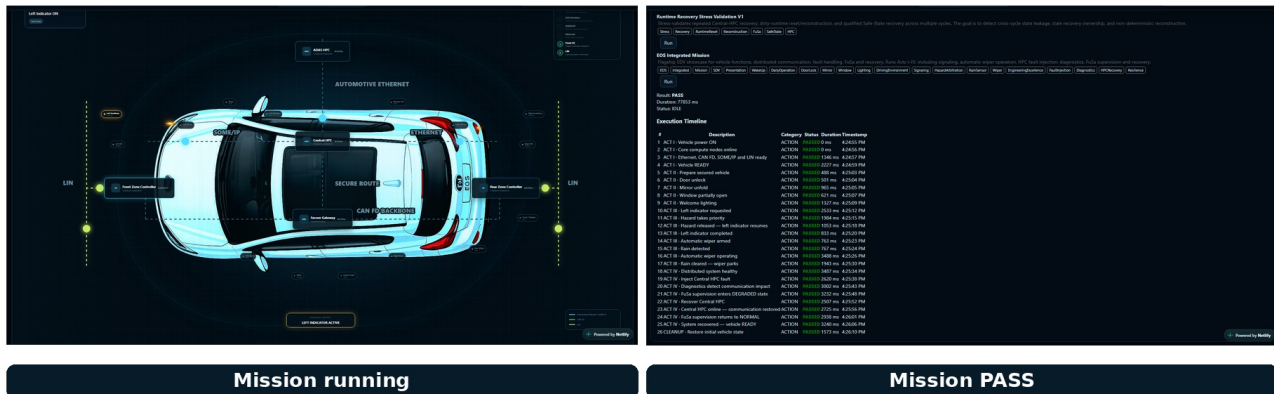
- request-rejection semantics;
- stale resolved mode;
- physical actuation;
- late-response evidence;
- FuSa contamination.

This validates that reset/quiescence can terminate old asynchronous work without allowing it to mutate the new runtime generation.

10.12 Feature-Scenario Review Matrix

Scenario	Main concern
DoorLock Discrete Actuator Scenario V1	Discrete transition semantics
FuelFlap Discrete Actuator Scenario V1	Discrete transition semantics
Window Continuous Actuator Scenario V3	Continuous motion and STOP
Mirror Continuous Actuator Scenario V2	Continuous motion and STOP
TurnLight Signaling Pattern Scenario V1	Signaling/arbitration
TurnLight Cancellation Validation V1	Distributed request cancellation
Wiper Periodic Motion Pattern Scenario V1	Periodic motion and modes
Wiper Physical Motion Validation V1	Physical movement correctness
Wiper Periodic Motion Fault Validation V1	Actuator faults
Wiper Periodic Motion Network Validation V1	AUTO network dependency/recovery
Wiper Cancellation Validation V1	Generation-safe cancellation

11. Runtime and Recovery Scenarios



Integrated runtime validation: execution and PASS evidence

These scenarios exercise system-level behavior beyond one vehicle function: node recovery, distributed-service cancellation, reset/reconstruction and repeated release-gate stress.

11.1 HPC Recovery Scenario

Purpose

Validate repeated Central HPC offline/online recovery, feature capability restoration, runtime orchestration and E2E stability.

Main sequence

The scenario:

- powers the vehicle;
- activates Turn Light and Wiper behavior;
- takes Central HPC offline;
- restores Central HPC;
- repeats the offline/online cycle;
- exercises Hazard;
- powers the vehicle off;
- validates TurnLight and Wiper capability/operation.

What to observe

- Topology node state;
- service availability;
- TurnLight and Wiper capability;
- communication events;
- diagnostics;
- final feature operational state.

This scenario is also used by the **Central HPC Fault & Recovery Demo Story**.

11.2 ADAS HPC Recovery Scenario

Purpose

Validate technical ADAS HPC recovery while Safe State remains intentionally latched.

Main sequence

- set ignition on;
- verify PredictiveLightingService availability;
- request predictive lighting;
- take ADAS HPC offline;
- verify PredictiveLightingService failure;
- make a request while the node is offline;
- restore ADAS HPC;
- verify PredictiveLightingService recovery;

- verify BrakeDecisionService recovery;
- verify Safe State remains latched after technical recovery;
- request predictive lighting again.

Key lesson

```
node/service technical recovery
    ≠
automatic Safe-State release
```

11.3 Distributed Service Cancellation Validation V1

Purpose

Validate active termination of an outstanding distributed service request during lifecycle quiescence.

Expected result

The cancelled request must settle as lifecycle cancellation without producing:

- timeout;
- retry;
- late-response classification;
- final failure;
- FuSa contamination.

This scenario validates the generic distributed-service cancellation mechanism independent of a specific BCM feature.

11.4 Runtime Reset Quiescence Validation V1

Purpose

Validate reset ownership of execution-generation transition and quiescence.

Main validation points

- establish `IGNITION_ON`;
- begin a pending TurnLight request;
- reset while the request is pending;
- verify canonical reset lifecycle order;
- verify reset finishes `READY / IDLE`;
- verify the obsolete request does not become rejection semantics;
- verify it cannot mutate TurnLight state or lamps after reset;
- verify timeout/retry ownership is terminated;
- verify no false late response;
- verify no FuSa deadline contamination.

Key lesson

Old asynchronous work must not cross a runtime-generation boundary.

11.5 Runtime Reset Reconstruction Validation V1

Purpose

Validate deterministic reconstruction of service, dependency and feature lifecycle truth.

Main sequence

```
PolicyService AVAILABLE
TurnLight AVAILABLE
↓
PolicyService FAILED
↓
TurnLight DEGRADED
↓
runtime reset
↓
service reconstructed
REGISTERED → STARTING → AVAILABLE
↓
dependency evidence rebuilt
```

```

    ↓
TurnLight lifecycle reconstructed
    ↓
reset READY / IDLE

```

Key lesson

Reset does not merely clear state. It rebuilds current truth from authoritative sources.

11.6 Runtime Reset FuSa Reconstruction Validation V1

Purpose

Validate that runtime reset clears transient FuSa ownership without recreating obsolete safety reactions during reconstruction.

Main checks

- start from clean FuSa state;
- fail PolicyService;
- verify SafetySupervisor transient state;
- verify DegradedModeManager ownership;
- reset runtime;
- verify transient FuSa state cleared;
- verify service reconstruction does not recreate obsolete reactions;
- verify reset finishes [READY](#) / [IDLE](#).

11.7 Runtime Reset FTTI Cancellation Validation V1

Purpose

Validate reset while an FTTI window is outstanding.

Main checks

- start from normal runtime state;
- open an FTTI window at the reaction-execution boundary;
- confirm the window is outstanding;
- reset runtime;
- confirm obsolete FTTI ownership is cancelled;
- verify BrakeDecisionService reconstructs to [AVAILABLE](#);
- verify reset finishes [READY](#) / [IDLE](#).

Key lesson

Timing supervision from the previous runtime generation must not fire after reset.

11.8 Runtime Recovery Stress Validation V1

Purpose

Stress repeated recovery and reconstruction to detect cross-cycle leakage or non-determinism.

The scenario performs three repeated cycles in each of the following areas:

Central HPC recovery

Repeated offline/online recovery while checking feature restoration.

Dirty-runtime reset/reconstruction

Repeated reconstruction from disturbed runtime conditions while verifying canonical baseline state.

Qualified Safe-State recovery

Repeated ASIL-D failure/recovery cycles verifying that technical recovery alone does not release Safe State and that explicit authorization/request is required.

Final assertion

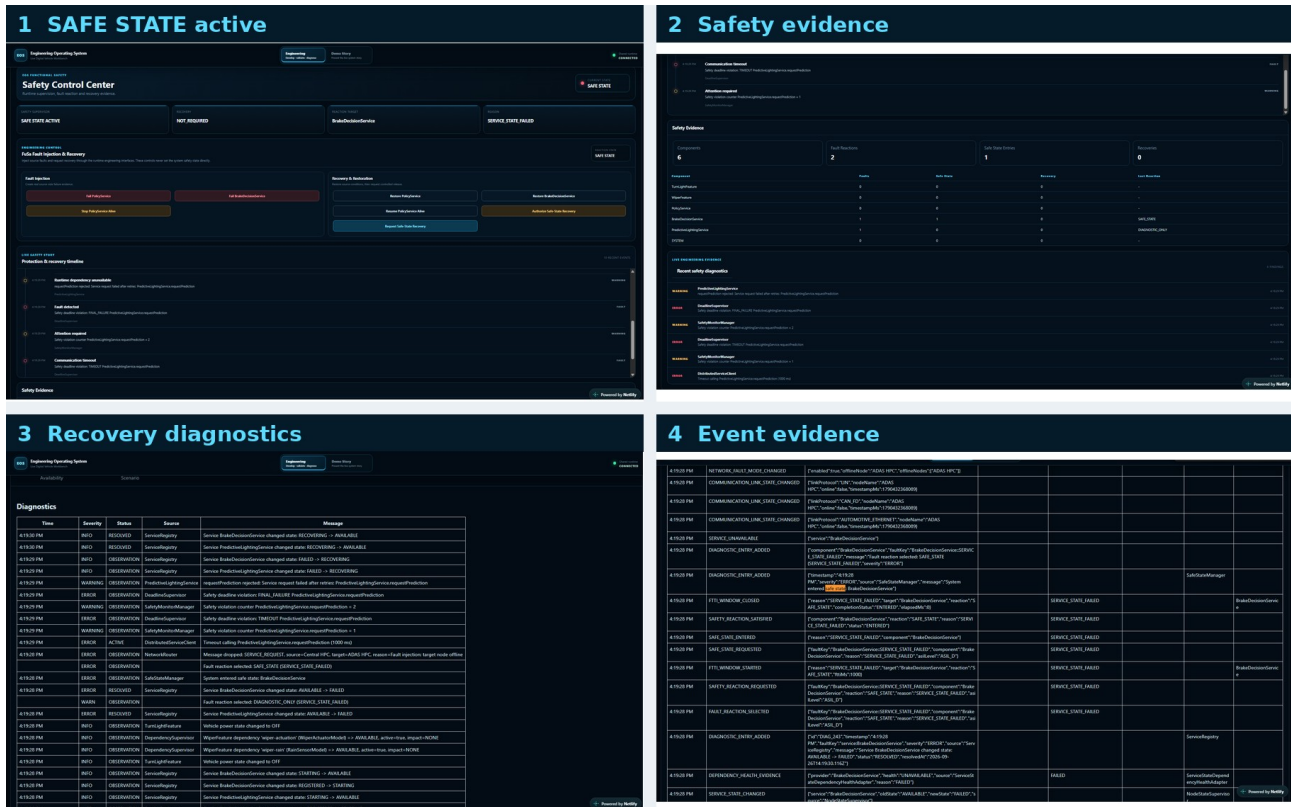
The scenario finishes by verifying the overall runtime is nominal after all stress cycles.

This scenario is particularly useful as a release-regression check because it looks for state that incorrectly survives one cycle and contaminates the next.

11.9 Runtime Scenario Review Matrix

Scenario	Main concern
HPC Recovery Scenario	Central HPC recovery / E2E stability
ADAS HPC Recovery Scenario	ADAS recovery with Safe-State latch
Distributed Service Cancellation Validation V1	Generic async cancellation
Runtime Reset Quiescence Validation V1	Generation boundary / quiescence
Runtime Reset Reconstruction Validation V1	Service/dependency/feature reconstruction
Runtime Reset FuSa Reconstruction Validation V1	FuSa transient-state reconstruction
Runtime Reset FTTI Cancellation Validation V1	Timing-window cancellation
Runtime Recovery Stress Validation V1	Repeated recovery / release-gate stress

12. Functional Safety Scenarios



Functional Safety evidence across fault and recovery states

The Functional Safety scenarios validate fault reaction, escalation, Safe-State behavior and controlled recovery. They are engineering scenarios inside the emulator and are not product-safety certification evidence.

12.1 Safe State Reaction Scenario

Purpose

Validate the ASIL-D service-failure reaction.

Sequence

```

IGNITION_ON
↓
BrakeDecisionService FAILED
↓
service failure evidence
↓
safety reaction
↓
SAFE STATE active

```

Expected result

PASS confirms that the BrakeDecisionService failure triggers the expected Safe-State reaction.

12.2 Safe State Recovery Scenario

Purpose

Validate qualified Safe-State recovery.

Sequence and expected behavior

1. Set ignition on.
2. Fail BrakeDecisionService.
3. Verify Safe State active.

4. Request recovery while escalation is active.
5. Verify recovery denied.
6. Authorize Safe-State recovery.
7. Request recovery while the initiating fault is still active.
8. Verify recovery denied.
9. Recover BrakeDecisionService.
10. Verify technical recovery alone does **not** release Safe State.
11. Request the now-authorized Safe-State recovery.
12. Verify recovery completes.

This scenario is the clearest executable demonstration of:

```
technical recovery
  ≠
safety authorization
  ≠
recovery request
```

All three concepts are separate.

12.3 Safety Supervisor Threshold Scenario

Purpose

Validate persistent-fault escalation and recovery-lockout thresholds.

Main sequence

The scenario repeatedly fails and recovers BrakeDecisionService, then verifies that repeated fault evidence crosses the configured persistent-fault threshold.

It also makes repeated recovery requests while escalation is active and verifies recovery-lockout escalation.

What to observe

- Safety timeline;
- safety evidence counters;
- diagnostics;
- Scenario Execution Timeline.

12.4 Feature Impact Escalation Scenario

Purpose

Validate safety impact originating from affected features rather than directly from an ASIL-D service reaction alone.

Main sequence

- set ignition on;
- fail PolicyService;
- verify TurnLightFeature impact;
- verify WiperFeature impact;
- verify feature reaction selection;
- verify feature-impact escalation;
- verify severity assessment.

V1.0 safety contracts define:

- TurnLightFeature as ASIL-B with a degraded-mode reaction for PolicyService impact;
- WiperFeature as ASIL-A with a degraded-mode reaction for PolicyService impact.

Engineering meaning

One provider fault can affect more than one feature. The safety architecture keeps feature impact, reaction selection and escalation as explicit evidence.

12.5 Safety Profiles Used by V1.0

The V1.0 Release Candidate contains the following default safety profiles:

Component	Safety classification	Safety relevant
BrakeDecisionService	ASIL-D	Yes

Component	Safety classification	Safety relevant
PolicyService	ASIL-B	Yes
TurnLightFeature	ASIL-B	Yes
WiperFeature	ASIL-A	Yes
PredictiveLightingService	QM	No

The generic fault-reaction policy maps:

- QM → diagnostic-only;
- ASIL-A / ASIL-B → degraded mode;
- ASIL-C / ASIL-D → Safe State.

Feature-specific safety contracts add explicit feature-impact behavior where required.

13. EOS Integrated Mission

EOS Integrated Mission is the flagship end-to-end scenario in BCM Emulator V1.0.

Its purpose is to demonstrate vehicle functions, distributed communication, fault handling, Functional Safety and recovery as one coherent live system story.

The V1.0 mission contains four Acts.

13.1 How to Run the Mission

Open:

Engineering → **Runtime** → **Scenario validation**

Select:

EOS Integrated Mission

and choose:

Run scenario

The mission uses the same Scenario Runner as all other executable scenarios, so a canonical runtime reset is performed automatically before the mission begins.

The mission also drives presentation focus between relevant Demo Story areas while it executes.

For the strongest presentation, keep **Demo Story** available and follow the automatically selected presentation areas.

13.2 Act I — Wake Up

Act I establishes the vehicle and computing baseline.

The mission performs:

1. **Vehicle power ON**
2. **Core compute nodes online**
 - Central HPC
 - ADAS HPC
3. **Ethernet, CAN FD, SOME/IP and LIN ready**
4. **Vehicle READY**

The protocol-readiness step is an observation hold. It does not fabricate communication events; runtime evidence remains the source of truth.

What to observe

- vehicle power;
- topology nodes;
- communication/runtime evidence;
- overall system state.

13.3 Act II — Daily Operation

Act II demonstrates ordinary body-control interactions.

The sequence is:

1. **Prepare secured vehicle**
 - Door Lock request
2. **Door unlock**
3. **Mirror unfold**
4. **Window partially open**
5. **Welcome lighting**

V1.0 does not contain a dedicated Welcome Light feature. The mission uses a short Hazard pulse through the real signaling runtime to create the welcome-lighting presentation without adding a separate feature.

What to observe

- Live Feature widgets;
- Door Lock state;
- Mirror movement;
- Window position;

- signaling output.

13.4 Act III — Driving & Environment

Act III combines signaling arbitration and automatic Wiper behavior.

Signaling sequence

```

Left indicator requested
      ↓
Hazard takes priority
      ↓
Hazard released
      ↓
Left indicator resumes
      ↓
Left indicator completed

```

Wiper sequence

```

AUTO armed
      ↓
rain detected
      ↓
automatic Wiper operates
      ↓
rain cleared
      ↓
Wiper parks

```

This Act demonstrates both request arbitration and conditional runtime dependencies.

13.5 Act IV — Engineering Excellence

Act IV introduces a Central HPC failure and follows the runtime through diagnosis and recovery.

The sequence is:

1. Distributed system healthy
2. Inject Central HPC fault
3. Diagnostics detect communication impact
4. FuSa supervision enters DEGRADED state
5. Recover Central HPC
6. Central HPC online — communication restored
7. FuSa supervision returns to NORMAL
8. System recovered — vehicle READY

The observation steps do not create presentation-only diagnostics or safety states. They wait for and display evidence produced by the shared runtime.

What to observe

Follow:

```

Topology
      ↓
Diagnostics
      ↓
Safety
      ↓
Topology
      ↓
Safety
      ↓
Topology

```

This mirrors the mission's presentation focus.

13.6 Mission Cleanup

The mission contains an explicit cleanup step, including restoration of:

- rain input;
- Wiper state;
- Hazard;

The final cleanup restores the presentation to a predictable post-mission condition.

This cleanup is distinct from the canonical pre-scenario reset.

13.7 What the Mission Demonstrates

The mission is valuable because it connects several independent engineering concepts into one runtime sequence:



It is therefore the recommended final demonstration after a user has already explored individual features and engineering views.

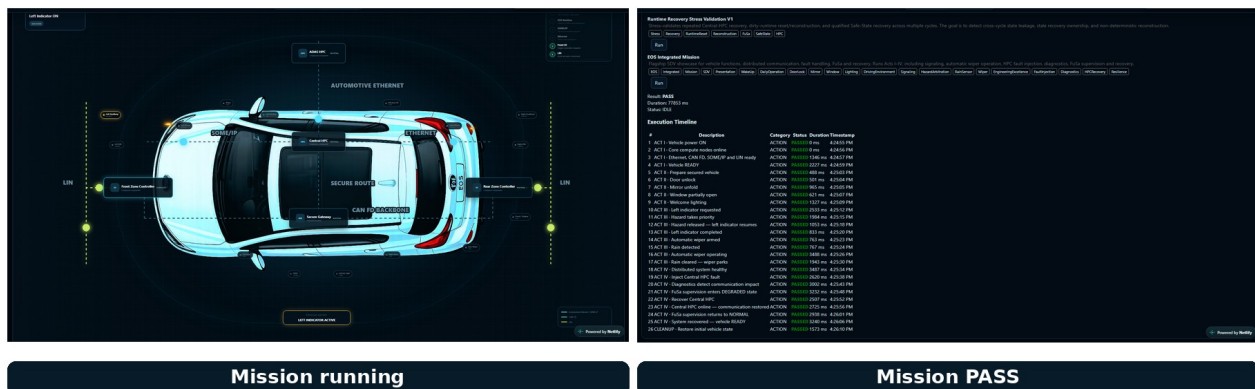


Figure 13-1 - EOS Integrated Mission: running and completed states

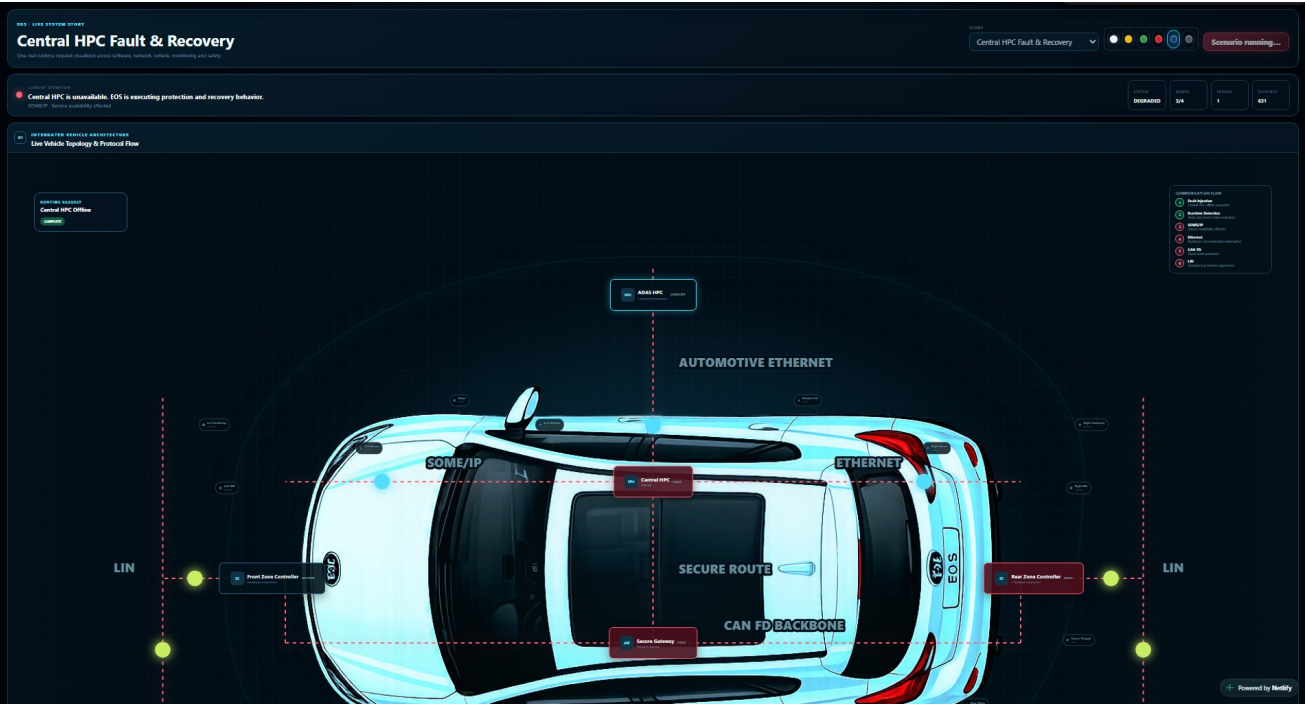


Figure 13-2 - Central HPC fault visible in Demo Story

14. Demo Story

The **Demo Story** workspace is the presentation-oriented view of the live BCM runtime. It does not create a separate simulation.

14.1 Purpose

Engineering view answers:

What is happening inside the runtime and why?

Demo Story answers:

How can the same live behavior be explained clearly as an end-to-end system story?

The workspace combines:

- live vehicle topology;
- protocol flow;
- communication inspection;
- live body-feature widgets;
- monitoring/diagnostics;
- Functional Safety;
- execution-story stages.

14.2 Story Controls

The Demo Story header provides:

- **Story** selector;
- vehicle color selector;
- **Run Live Story**;
- story-specific running/stop state.

V1.0 contains two selectable stories:

- **Left Indicator Execution**
- **Central HPC Fault & Recovery**

14.3 Left Indicator Execution

This story traces a real Left Indicator request through the live runtime.

The story stages include:

- Operator Request;
- EOS Runtime;
- SOME/IP;
- Ethernet;
- Front ZC;
- LIN;
- Vehicle Output;
- Monitoring;
- Safety.

The story powers the vehicle to **IGNITION_ON** if necessary and then activates the Left Indicator.

Completion is derived from runtime evidence such as communication activity and physical lamp state.

After completion, the final architecture is held briefly for inspection and the story then turns the Left Indicator off.

The operator can use **Stop Story** while this presentation is running.

14.4 Central HPC Fault & Recovery

This story uses the real **HPC Recovery Scenario**.

It visualizes stages including:

- Fault Injection;
- Runtime Detection;
- SOME/IP;
- Ethernet;
- CAN FD;
- LIN;
- System Protection;

- Automatic Recovery;
- Evidence.

While the underlying scenario is executing, the Demo Story button displays:

Scenario running...

and is disabled for that story.

Important distinction

The Demo Story can describe the system as **PROTECTED** when a Central HPC fault is present, but the underlying detailed safety state must still be read from the shared runtime evidence.

14.5 Live Vehicle Topology & Protocol Flow

The main Demo Story system map shows:

- current vehicle package;
- vehicle power;
- topology nodes;
- offline-node state;
- communication stages;
- active protocol badges;
- execution progress.

Protocol presentation includes:

- Automotive Ethernet;
- SOME/IP;
- CAN FD where applicable;
- LIN.

Packet/protocol motion is driven by real communication events.

14.6 Live Route & Protocol Inspector

Demo Story includes the **Communication Runtime — Live Route & Protocol Inspector**.

For the currently selected communication message it shows:

- Event;
- Message ID;
- Route reason;
- Estimated latency;
- Deployment path;
- live hop progress;
- protocol stack for each hop;
- recent traffic.

Message states include:

- ROUTED
- IN_PROGRESS
- DELIVERED
- FAILED

Hop states include:

- WAITING
- ACTIVE
- COMPLETE
- FAILED

This view is especially useful for demonstrating that a feature request is resolved through deployment-aware communication rather than being animated directly in the UI.

14.7 Live Feature Widgets

Demo Story contains clickable live widgets for:

- Wiper;
- Window;
- Mirror;
- Door Access.

These widgets act on the same shared runtime.

They are useful during presentations for demonstrating that Demo Story remains interactive and is not a prerecorded sequence.

14.8 Monitoring & Diagnostics

The Demo Story evidence card summarizes:

- active Errors;
- active Warnings;
- Events.

It also shows recent diagnostic entries derived from the story's own evidence window.

The story evidence is scoped to the current/most-recent Demo Story run rather than relying only on the globally capped Engineering event history.

14.9 Protection & Recovery

The Demo Story Safety card converts runtime safety status into presentation-oriented labels:

Runtime status	Demo Story presentation
NORMAL	NORMAL · SUPERVISED
DEGRADED	DEGRADED · SUPERVISED
SAFE STATE	SYSTEM PROTECTED

These are presentation labels. The shared runtime remains authoritative.

14.10 Execution Story

The final card shows the end-to-end story stages and their current status.

Stage status can be:

- WAITING
- ACTIVE
- COMPLETE
- FAULT

This allows an audience to follow the request from input through software, communication and vehicle output without reading the full Engineering Timeline.

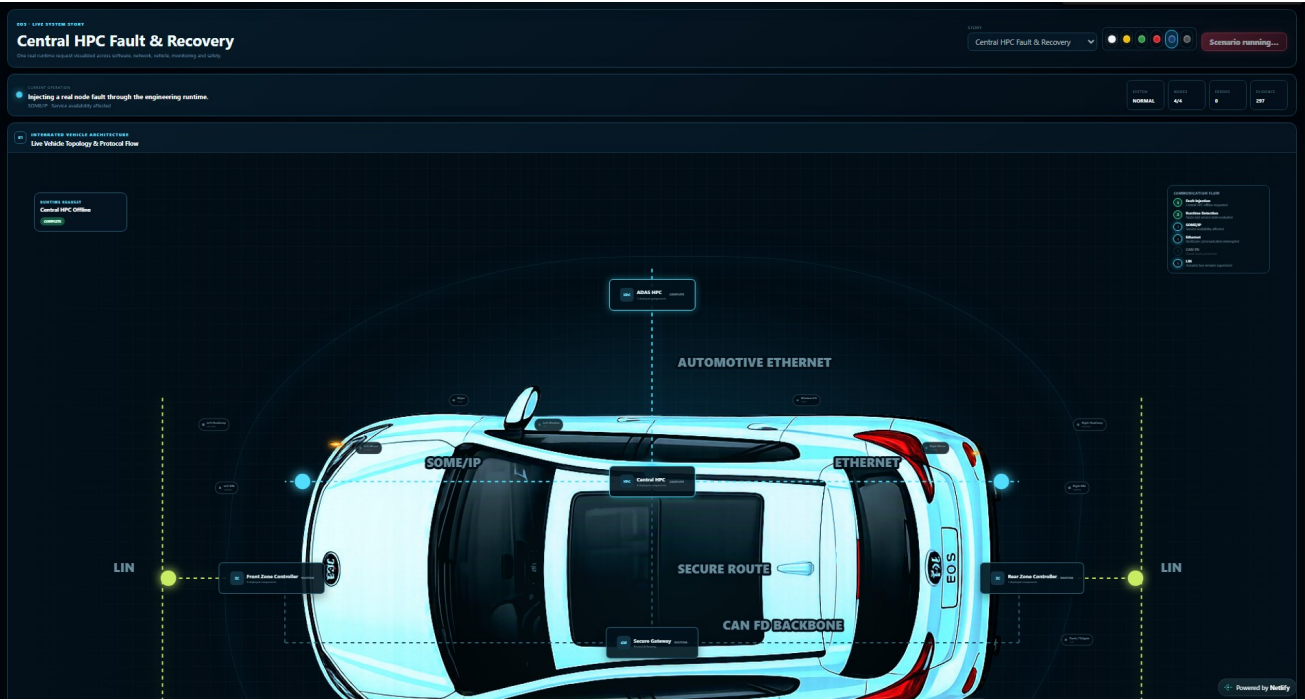


Figure 14-1 - Full Demo Story workspace

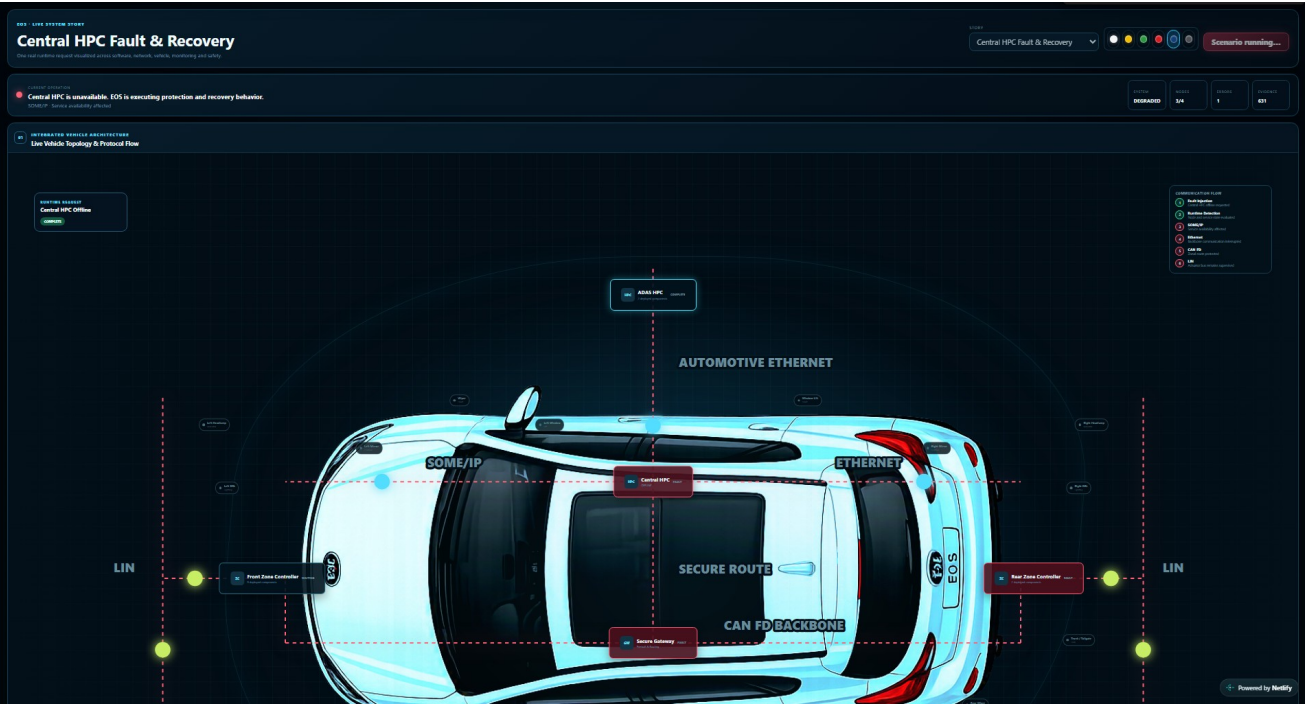
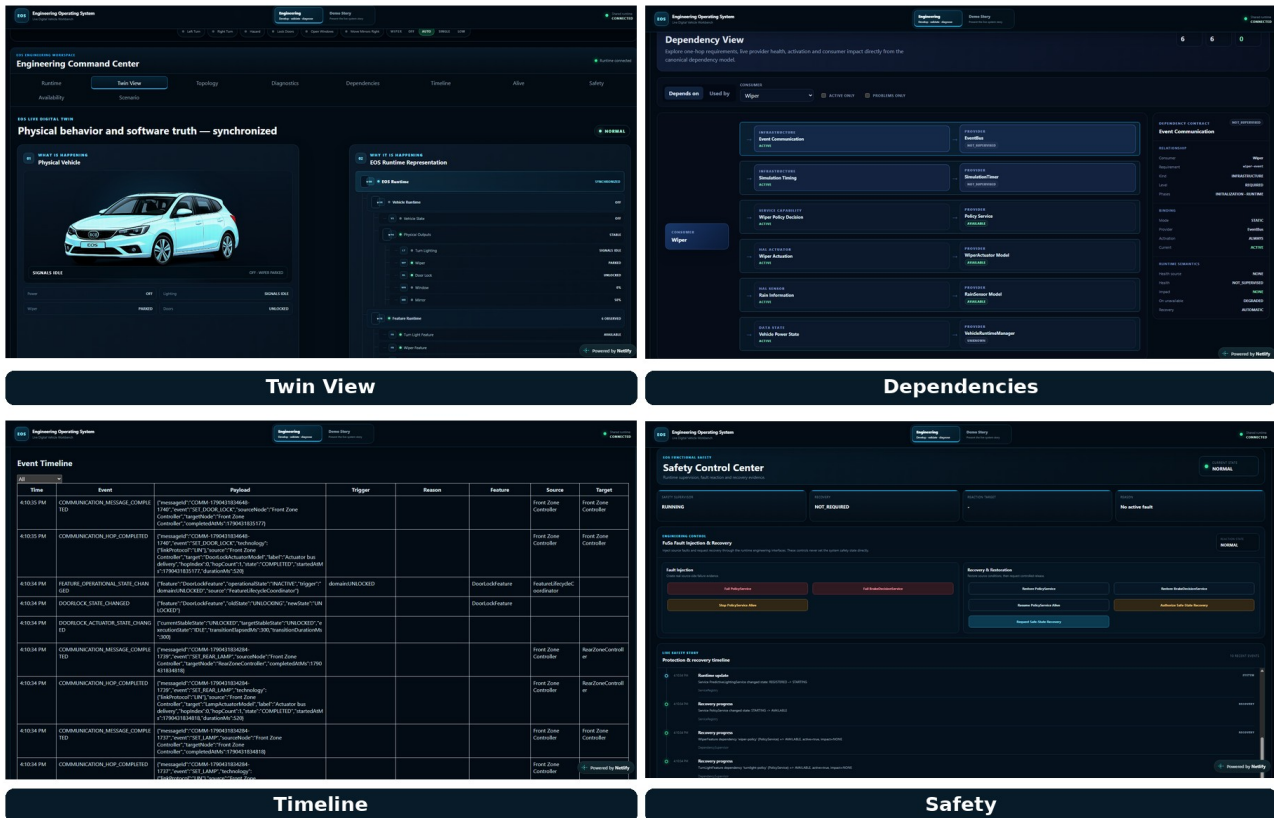


Figure 14-3 - Central HPC story during protected/degraded operation

15. Engineering Observability



Engineering observability: Twin View, Dependencies, Timeline and Safety

BCM Emulator V1.0 treats observability as part of the engineering system rather than as a decorative UI layer. The goal is to let a user correlate visible vehicle behavior with runtime evidence.

15.1 Twin View — Physical vs Runtime Truth

Twin View is the fastest way to compare:

WHAT IS HAPPENING
Physical Vehicle
↔
WHY IT IS HAPPENING
EOS Runtime Representation

Use it when:

- a visible actuator state looks unexpected;
- a feature has changed capability;
- signaling or Wiper behavior needs runtime correlation;
- a fault/recovery sequence should remain synchronized.

15.2 Dependency View — Cause and Impact

Use **Depends on** when starting from a consumer.

Use **Used by** when starting from a provider.

Important fields include:

- consumer;
- requirement/capability;
- provider;
- kind;
- requirement level;
- lifecycle phase;
- binding mode;
- activation;
- health source;
- health;

- impact;
- recovery policy.

Filters:

- **Active only**
- **Problems only**

For root-cause analysis, the Problems filter is often the quickest way to isolate an unhealthy active relationship.

15.3 Diagnostics — Recorded Fault Evidence

Diagnostics show:

- Time;
- Severity;
- Status;
- Source;
- Message.

Diagnostic status can distinguish current and historical evidence.

Do not interpret the presence of an old diagnostic row as proof that the fault remains active.

The V1.0 diagnostic model can retain evidence after current runtime state has recovered.

15.4 Event Timeline — Sequence of Evidence

Timeline provides event-level chronological evidence.

Filter categories are:

- All;
- Input;
- Business;
- Communication;
- Availability;
- Safety.

The table exposes:

- Time;
- Event;
- Payload;
- Trigger;
- Reason;
- Feature;
- Source;
- Target.

Use Timeline when the key question is:

What happened first, and what followed?

Long payloads wrap rather than forcing the rest of the table into unreadable narrow columns.

15.5 Alive — Liveness Evidence

Alive View reports:

- Component;
- Status;
- Last Alive;
- Elapsed.

Primary statuses are:

- **ALIVE**
- **UNRESPONSIVE**

This is the correct view for heartbeat/liveness evidence.

A higher-level safety or feature state may react to liveness loss, but Alive View remains the direct observation of supervision itself.

15.6 Availability — Current State Across Ownership Domains

Availability separates:

- Nodes;

- Runtime Services;
- Deployed Runtime Components;
- Runtime Features.

This is a key current-state view after fault injection or recovery.

If a historical event says a service failed but Availability now shows the reconstructed current state, both can be correct because they answer different questions.

15.7 Topology — Deployment and Communication Context

Topology helps answer:

- where a component is deployed;
- which node is offline;
- how communication moves through the vehicle;
- where a fault was injected.

Use it together with the Communication Inspector when the investigation concerns distributed request routing.

15.8 Communication Inspector — Route-Level Evidence

The Communication Inspector provides deployment-aware communication evidence.

For a message, inspect:

```

event
  ↓
deployment path
  ↓
hop sequence
  ↓
application / transport / network / link protocols
  ↓
delivery or failure

```

The inspector keeps a short recent-traffic history and lets the user select a previous message in that window.

15.9 Safety — Reaction Evidence

Safety combines:

- current system state;
- reaction target/reason;
- recovery state;
- engineering fault/recovery controls;
- safety timeline;
- safety evidence counters;
- recent safety diagnostics.

Use it when the question is not merely "what failed?" but:

What protection reaction did the system select, and has that reaction been released?

15.10 Correlating Evidence

A powerful BCM investigation uses several views together.

Example:

```

Symptom:
TurnLight does not behave as expected
  ↓
Availability:
TurnLight = DEGRADED
  ↓
Dependencies:
PolicyService relationship unhealthy
  ↓
Topology:
Central HPC offline
  ↓
Diagnostics:
node/service fault evidence
  ↓
Timeline:
ordering of node, service, dependency and feature events
  ↓

```

Safety:
degraded reaction evidence

No single panel is expected to answer every engineering question.

15.11 Current State vs Historical Evidence

Use this rule throughout the emulator:

Current-state views answer "what is true now"; history views answer "what happened".

Typical current-state views:

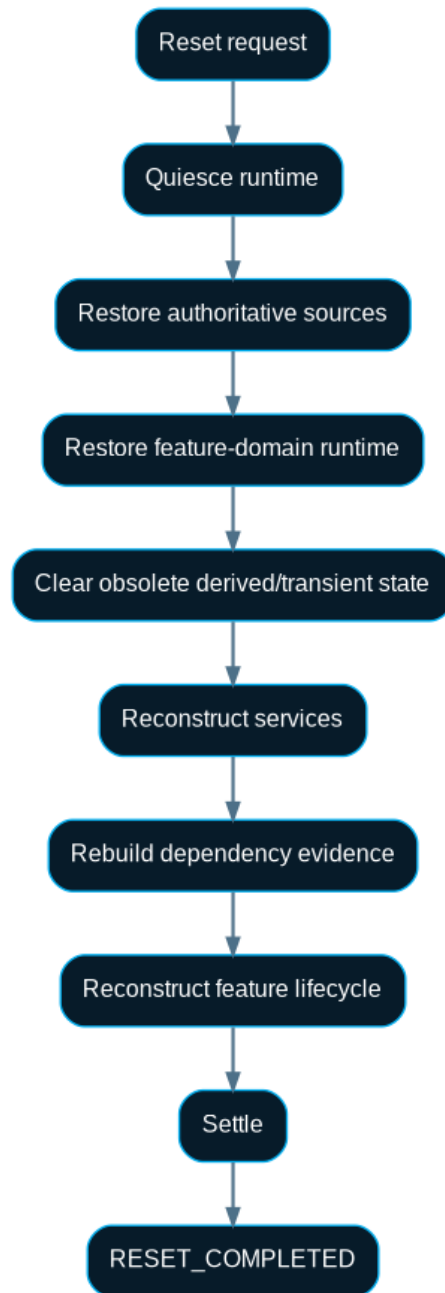
- Runtime;
- Twin View;
- Dependencies;
- Availability;
- Safety;
- Alive;
- Topology.

Typical history/evidence views:

- Timeline;
- Diagnostics;
- Scenario Execution Timeline;
- Demo Story run evidence.

The distinction prevents recovered systems from being misread as still failed simply because historical evidence remains visible.

16. Reset and Reconstruction



Canonical reset and reconstruction flow

Runtime reset is one of the most important V1.0 mechanisms because scenarios, asynchronous communication, dependencies and Functional Safety all retain state that must be reconstructed coherently.

16.1 What Reset Means in V1.0

A canonical runtime reset is not:

```
clear UI
```

and it is not:

```
set everything to healthy
```

It is a controlled lifecycle:

```

quiesce old execution
↓
restore authoritative sources
↓
restore feature-domain runtime
↓

```

```

clear obsolete derived/transient knowledge
↓
reconstruct services
↓
republish authoritative evidence
↓
reconstruct feature lifecycle
↓
settle

```

The goal is deterministic current truth.

16.2 Reset State and Phases

Reset state:

- `READY`
- `RESETTING`
- `FAILED`

Reset phases:

1. `QUIESCE`
2. `RESTORE_SOURCE`
3. `RESTORE_FEATURE_RUNTIME`
4. `RECONSTRUCT`
5. `REBUILD_EVIDENCE`
6. `SETTLE`
7. return to `IDLE`

Lifecycle events such as `RESET_STARTED`, phase events, `RESET_COMPLETED` and `RESET_FAILED` make the process observable.

16.3 Quiescence

The first responsibility is to stop obsolete asynchronous ownership.

V1.0 quiesces:

- outstanding distributed-service requests;
- pending network deliveries belonging to the previous execution generation.

A cancelled request is settled as lifecycle cancellation.

It should not later appear as:

- timeout;
- retry;
- final failure;
- genuine late response.

16.4 Restore Authoritative Sources

The reset restores baseline source state for areas including:

- network fault configuration;
- network latency configuration;
- vehicle power;
- Rain Sensor;
- lamp actuators;
- Wiper actuator;
- Door Lock actuator;
- Fuel Flap actuator;
- Window actuator;
- Mirror actuator.

These are authoritative source conditions from which derived runtime knowledge can later be rebuilt.

16.5 Restore Feature-Domain Runtime

Turn Light and Wiper contain domain/runtime state that is explicitly restored before reconstruction proceeds.

This prevents old feature-domain execution from leaking into the new runtime generation.

16.6 Clear Obsolete Derived and Transient State

Before evidence is replayed, reset clears transient ownership in:

- runtime request diagnostics;
- dependency health/supervision;
- feature-dependency coordination;
- feature lifecycle state;
- lifecycle-managed service state;
- FTTI monitoring;
- safety recovery;
- safety monitoring;
- Safety Supervisor;
- Degraded Mode;
- Safe State;
- Alive supervision.

Persistent infrastructure, registrations, contracts and subscriptions remain.

16.7 Reconstruct Services

Lifecycle-managed services are reconstructed through:

```
REGISTERED
  ↓
STARTING
  ↓
AVAILABLE
```

These service-state changes become fresh provider evidence for dependent components.

16.8 Rebuild Dependency Evidence

After service reconstruction, authoritative domain/HAL state is republished.

Existing dependency adapters and supervision then rebuild dependency health from current evidence.

This avoids reconstructing state from old event history.

16.9 Reconstruct Feature Lifecycle

Features are lifecycle-initialized only after dependency evidence has been rebuilt.

This ordering avoids a false transient `AVAILABLE` state before degraded or unavailable dependency reasons are known.

The key sequence is:

```
service evidence
  ↓
source/HAL evidence
  ↓
dependency truth
  ↓
feature lifecycle truth
```

16.10 Settle and Complete

Once reconstruction is complete:

- execution context is marked ready;
- reset state becomes `READY`;
- phase becomes `IDLE`;
- `RESET_COMPLETED` is published.

If a phase fails, reset enters `FAILED` and records the failed phase and reason.

16.11 When Users Encounter Reset

The Public Preview does not expose a general full-runtime Reset button.

Users encounter canonical reset primarily through:

- scenario startup;
- reset validation scenarios;
- repeated stress/regression scenarios.

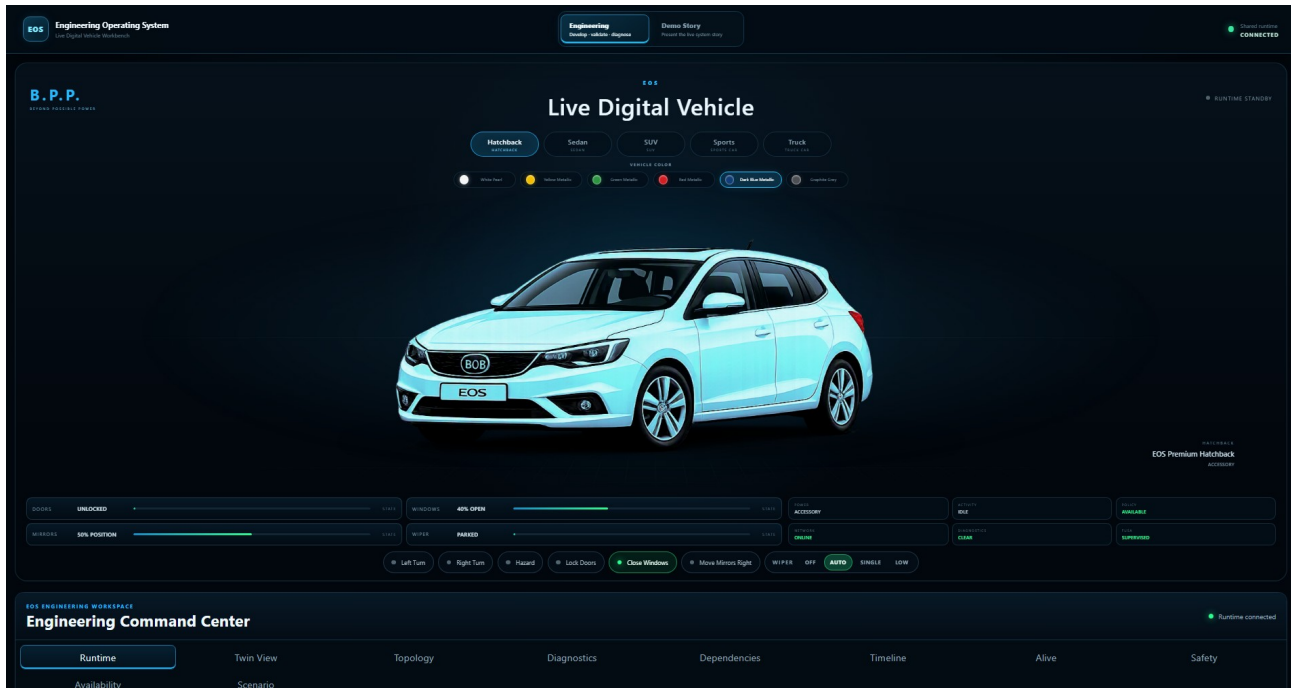
This is intentional: reset is an engineering lifecycle operation, not a cosmetic UI action.

16.12 Reset vs Recovery

Keep the distinction clear:

Operation	Purpose
Restore node/service/fault source	Recover one technical condition
Authorize/request Safe-State recovery	Release a qualified safety reaction
Recover All in Topology	Restore network-fault controls
Vehicle OFF	Change vehicle power state
Canonical runtime reset	Reconstruct the complete runtime baseline

17. Typical User Workflows



Engineering workspace used across typical user workflows

This section combines the previous concepts into practical workflows.

17.1 Explore a Vehicle Feature

1. Open **Engineering**.
2. Select **Runtime**.
3. Set the required vehicle power state.
4. Operate the feature from Live Digital Vehicle or Runtime Quick Commands.
5. Observe the vehicle response.
6. Open **Twin View**.
7. Open **Availability**.
8. Open **Timeline**.
9. Open **Dependencies**.
10. Run the corresponding feature scenario.

Use this workflow for Door Lock, Fuel Flap, Window, Mirror, Turn Light/Hazard or Wiper.

17.2 Investigate a Dependency Failure

1. Establish a normal operating state.
2. Open **Dependencies** and select the consumer.
3. Record the active provider relationship.
4. Inject a fault affecting that provider.
5. Enable **Problems only**.
6. Inspect dependency Health and Impact.
7. Open **Availability** and inspect feature capability.
8. Open **Timeline** to reconstruct the sequence.
9. Restore the provider/source condition.
10. Confirm the dependency and feature recover.

Wiper AUTO + PolicyService/Rain Sensor and TurnLight + PolicyService are useful examples.

17.3 Investigate a Node / Network Failure

1. Open **Topology**.
2. Trigger a vehicle function to establish normal communication evidence.
3. Inject **Central HPC Offline** or **Front ZC Offline**.

4. Observe node state and communication behavior.
5. Open **Availability**.
6. Open **Diagnostics**.
7. Open **Dependencies**.
8. Open **Safety**.
9. Restore using **Recover All** or an appropriate scenario.
10. Verify current-state recovery.

For a deterministic guided version, run **HPC Recovery Scenario**.

17.4 Observe Degradation and Recovery

1. Open **Safety**.
2. Fail **PolicyService**.
3. Observe degraded-mode/safety evidence.
4. Inspect TurnLight and Wiper impact in **Availability**.
5. Inspect PolicyService relationships in **Dependencies**.
6. Restore **PolicyService**.
7. Observe fresh availability/alive evidence and reaction release.
8. Confirm the current system returns to nominal operation.

For a validated version, run **Feature Impact Escalation Scenario** or **Runtime Reset FuSa Reconstruction Validation V1**, depending on the question.

17.5 Run a Safe-State Experiment

The safest reproducible user path is the **Safe State Recovery Scenario**.

For manual exploration:

1. Open **Safety**.
2. Fail **BrakeDecisionService**.
3. Confirm **SAFE STATE**.
4. Try **Request Safe-State Recovery**.
5. Observe denial while escalation is active.
6. Select **Authorize Safe-State Recovery**.
7. Request recovery while the service is still failed.
8. Observe denial because the initiating fault remains active.
9. Select **Restore BrakeDecisionService**.
10. Confirm Safe State remains latched.
11. Select **Request Safe-State Recovery**.
12. Confirm Safe-State exit.

Use Timeline and Diagnostics to inspect each decision.

17.6 Run the Complete System Demonstration

For an end-to-end demonstration:

1. Select an appropriate vehicle package/color.
2. Open **Demo Story** in preparation for presentation.
3. Run **EOS Integrated Mission**.
4. Follow the presentation focus through Acts I–IV.
5. Observe normal features, communication, fault injection, diagnostics, degradation and recovery.
6. After mission completion, inspect Engineering Timeline and Scenario result.

This is the most comprehensive V1.0 demonstration.

17.7 Perform a Regression Confidence Check

A compact confidence workflow is:

1. Run one feature scenario from each pattern family:
 - DoorLock;
 - Window;
 - TurnLight;
 - Wiper.
2. Run **HPC Recovery Scenario**.

3. Run **Safe State Recovery Scenario**.
4. Run **Runtime Reset Reconstruction Validation V1**.
5. Run **Runtime Recovery Stress Validation V1**.
6. Repeat selected scenarios in a different order.

Because every scenario establishes its own canonical baseline, order-dependent failures are meaningful evidence of leakage or reconstruction problems.

18. BCM Emulator V1.0 Scope and Limitations

V1.0 has a frozen feature/architecture scope and is now represented by the current **Release Candidate** baseline. Understanding those boundaries prevents the Public Preview from being interpreted as something it is not.

18.1 V1.0 Feature Scope

The body-control feature portfolio is:

- Door Lock;
- Fuel Flap;
- Window;
- Mirror;
- Turn Light / Hazard;
- Wiper.

The portfolio intentionally covers several behavior patterns rather than attempting to model every BCM function.

18.2 Vehicle Presentation Scope

V1.0 contains five visual vehicle packages:

- Hatchback;
- Sedan;
- SUV;
- Sports;
- Truck.

It provides six colors:

- White Pearl;
- Yellow Metallic;
- Green Metallic;
- Red Metallic;
- Dark Blue Metallic;
- Graphite Grey.

Vehicle package/color selection changes presentation, not runtime architecture.

18.3 Topology Scope

The V1.0 Release Candidate topology contains four principal compute nodes:

- Central HPC;
- ADAS HPC;
- Front Zone Controller;
- Rear Zone Controller.

The topology and deployment are engineering models used by the emulator; they are not a claim that a specific production vehicle uses this exact ECU layout.

18.4 Provider Binding Scope

V1.0 uses:

STATIC provider binding

A dependency requirement is associated with a predefined provider.

The runtime does not implement dynamic service discovery, primary/fallback provider rebinding or mode-dependent provider switching.

18.5 Communication Scope

V1.0 models deployment-aware communication and exposes protocol/route evidence including Automotive Ethernet, SOME/IP, CAN FD and LIN where defined by the communication route.

This is an engineering communication model, not a production vehicle communication stack.

18.6 Functional Safety Scope

V1.0 demonstrates:

- safety profiles;

- ASIL-oriented reaction policy;
- alive/deadline supervision;
- fault reaction selection;
- degraded mode;
- Safe State;
- FTTI evidence;
- recovery supervision;
- controlled Safe-State release;
- feature-impact escalation.

It is not:

- an ISO 26262 safety case;
- an ASIL certification artifact;
- production safety validation;
- evidence that the architecture satisfies a specific vehicle's safety goals.

18.7 Runtime Reset Scope

Canonical reset/reconstruction is implemented and used by scenario boundaries.

The Public Preview does not expose a generic operator "make everything healthy" button.

This is intentional because V1.0 treats reset as a lifecycle/reconstruction operation.

18.8 Scenario Scope

V1.0 registers 24 executable scenarios.

Scenarios validate the behavior they explicitly assert. A PASS should not be interpreted as proof of requirements or failure modes that the scenario does not exercise.

18.9 Browser Engineering Environment

The V1.0 implementation is a browser-based engineering emulator.

Timing and presentation therefore operate in a host/browser environment while modeling target-system behavior.

The Public Preview should not be interpreted as a real-time embedded target, HIL system or production ECU runtime.

18.10 Release Candidate Discipline

The V1.0 feature and architecture scope is frozen, while the current source package is the **V1.0 Release Candidate**.

The purpose is reproducibility without ignoring genuine release defects:

```
frozen feature / architecture scope
+
known implementation
+
known scenarios
+
known behavior
+
aligned documentation
=
repeatable Release Candidate
```

New architecture or feature work belongs to a later version.

Before Public Preview release, the Release Candidate may still receive a narrowly controlled correction when a genuine release blocker, externally visible inconsistency or release-quality defect is confirmed. Such a correction must not silently expand V1.0 scope.

19. Troubleshooting

This section focuses on observable V1.0 behavior and the controls available in the Public Preview.

19.1 A Scenario Does Not Start

Check:

1. Is another scenario already running?
2. Is the Run button disabled because the runner is busy?
3. Did the previous scenario finish?
4. Does the Scenario view show a reset/baseline failure?

Only one scenario can run at a time.

The runner performs a canonical reset before execution. If that reset fails, the scenario reports failure rather than running on an uncertain baseline.

19.2 A Feature Command Appears to Do Nothing

Check vehicle power requirements.

Typical V1.0 permissions include:

- Window: `ACCESSORY` or `IGNITION_ON`;
- Mirror: `ACCESSORY` or `IGNITION_ON`;
- Turn Light: powered operation;
- Wiper: `IGNITION_ON`.

Then check:

- Availability;
- Dependencies;
- Timeline for `FeatureRequestRejected`;
- Diagnostics.

A rejected request can be correct behavior when its operating preconditions are not satisfied.

19.3 Wiper AUTO Does Not Wipe

Check:

1. vehicle is `IGNITION_ON`;
2. Wiper mode is `AUTO`;
3. rain is detected;
4. PolicyService is available;
5. Rain Sensor dependency is healthy;
6. Central HPC/network path is not preventing the AUTO policy request.

Use **Dependencies** to inspect the conditional Wiper AUTO relationships.

19.4 A System Still Shows Historical Errors After Recovery

Check whether you are looking at:

- historical Diagnostics/Timeline;
- or current Availability/Safety/Dependency state.

Old entries are retained as engineering evidence.

A recovered system can therefore have historical ERROR rows while the corresponding current fault is no longer active.

19.5 The System Remains in Safe State After the Service Recovers

This can be expected.

For Safe-State recovery:

1. restore the initiating technical fault;
2. use **Authorize Safe-State Recovery**;
3. use **Request Safe-State Recovery**;
4. inspect Safety and Diagnostics for denial/release evidence.

Technical service recovery alone intentionally does not release Safe State.

19.6 A Node Is Online but a Feature Is Not Fully Available

Node state is only one layer.

Check:

- lifecycle-managed service state;
- dependency health;
- feature lifecycle reasons;
- actuator/sensor health;
- current safety reaction.

An online host does not prove that every dependency or feature has recovered.

19.7 Recover All Did Not Perform a Full Runtime Reset

This is expected.

Recover All is a Topology/network-fault recovery control. It restores network fault settings, offline nodes, packet loss and latency.

It is not the canonical Runtime Reset Coordinator.

Run another scenario if you need the scenario-owned canonical baseline.

19.8 Demo Story and Engineering Seem to Show the Same State

That is expected and is an important product behavior.

Both workspaces use the same shared runtime.

Demo Story is a different presentation, not a second simulation.

19.9 Demo Story Left Indicator Does Not Start Immediately

The story first normalizes any previously active Left Indicator request, opens a new evidence window, performs its short presentation boot/observation sequence and ensures ignition is ready.

This prevents evidence from a previous run from satisfying the new story.

19.10 The Scenario Result Is FAIL

Start with the failed step in:

Engineering → **Scenario** → **Execution Timeline**

Then correlate:

- Timeline;
- Diagnostics;
- Availability;
- Dependencies;
- Safety if applicable.

Do not manually force states merely to make the result green. A failed assertion is engineering evidence that should be investigated.

19.11 Returning to a Known Baseline

The most deterministic user method is:

1. allow any running scenario to complete;
2. select the next scenario;
3. run it;
4. let Scenario Runner perform the canonical pre-scenario reset.

For presentation-only network experiments, **Recover All** can restore the Topology fault settings, but it is not equivalent to full runtime reconstruction.

20. Appendix

20.1 State Terminology

Term	V1.0 values / examples
Vehicle power	OFF, ACCESSORY, IGNITION_ON
Feature capability	INITIALIZED, AVAILABLE, DEGRADED, UNAVAILABLE, FAILED
Feature operation	ACTIVE, INACTIVE
Service lifecycle	REGISTERED, STARTING, AVAILABLE, DEGRADED, FAILED, RECOVERING, UNAVAILABLE
Node state	ONLINE, OFFLINE
Dependency health	AVAILABLE, DEGRADED, UNAVAILABLE, UNKNOWN
Dependency impact	NONE, DEGRADED, UNAVAILABLE
Safety current-state UI	NORMAL, DEGRADED, SAFE STATE
Reset state	READY, RESETTING, FAILED
Scenario result	PASS, FAIL, CANCELLED
Scenario step	PENDING, RUNNING, PASSED, FAILED

20.2 Event Categories

The Engineering Timeline groups events into:

- **Input**
- **Business**
- **Communication**
- **Availability**
- **Safety**

Examples:

Input

Vehicle power, Rain Sensor, Left/Right Indicator and Hazard requests.

Business

Feature state, feature rejection, operational state, Wiper mode and arbitration decisions.

Communication

Service request/response, retries/timeouts, network delivery/drop, E2E evidence and diagnostic-entry publication.

Availability

Node state, service state, dependency health/activation and feature capability.

Safety

E2E violations, safety escalation, Safe State, FTTI, reaction selection/release and feature safety impact.

20.3 V1.0 Scenario Catalog

#	Scenario	Guide section
1	HPC Recovery Scenario	11
2	ADAS HPC Recovery Scenario	11
3	Safe State Reaction Scenario	12
4	Safe State Recovery Scenario	12
5	Safety Supervisor Threshold Scenario	12
6	Feature Impact Escalation Scenario	12
7	Window Continuous Actuator Scenario V3	10
8	Mirror Continuous Actuator Scenario V2	10
9	DoorLock Discrete Actuator Scenario V1	10
10	FuelFlap Discrete Actuator Scenario V1	10
11	TurnLight Signaling Pattern Scenario V1	10
12	TurnLight Cancellation Validation V1	10
13	Wiper Periodic Motion Pattern Scenario V1	10

#	Scenario	Guide section
14	Wiper Physical Motion Validation V1	10
15	Wiper Periodic Motion Fault Validation V1	10
16	Wiper Periodic Motion Network Validation V1	10
17	Wiper Cancellation Validation V1	10
18	Distributed Service Cancellation Validation V1	11
19	Runtime Reset Quiescence Validation V1	11
20	Runtime Reset Reconstruction Validation V1	11
21	Runtime Reset FuSa Reconstruction Validation V1	11
22	Runtime Reset FTTI Cancellation Validation V1	11
23	Runtime Recovery Stress Validation V1	11
24	EOS Integrated Mission	13

The table follows the scenario registration order in the V1.0 Release Candidate runtime.

20.4 Vehicle Packages and Colors

Packages

- EOS Premium Hatchback
- EOS Executive Sedan
- EOS Adventure SUV
- EOS Performance Coupe
- EOS Truck

Colors

- White Pearl
- Yellow Metallic
- Green Metallic
- Red Metallic
- Dark Blue Metallic
- Graphite Grey

20.5 Topology Summary

V1.0 principal nodes:

Node	Type
Central HPC	HPC
ADAS HPC	HPC
Front Zone Controller	Zone Controller
Rear Zone Controller	Zone Controller

Principal links:

```

  ADAS HPC —————→ Central HPC
  Front Zone Controller → Central HPC
  Central HPC —————→ Rear Zone Controller

```

The exact route/protocol evidence for a runtime request is best inspected in the Topology and Communication Inspector rather than inferred only from this simplified diagram.

20.6 Key Deployment Examples

Examples from the V1.0 Release Candidate deployment model include:

Central HPC

- PolicyService
- DiagnosticsManager
- VehicleRuntimeManager
- FeatureRuntimeManager
- WindowFeature / WindowActuatorModel
- MirrorFeature / MirrorActuatorModel

ADAS HPC

- PredictiveLightingService
- BrakeDecisionService

Front Zone Controller

- TurnLightFeature
- WiperFeature
- DoorLockFeature
- FuelFlapFeature
- front LampActuatorModel
- WiperActuatorModel
- RainSensorModel
- DoorLockActuatorModel
- FuelFlapActuatorModel

Rear Zone Controller

- RearLampActuatorModel
- RearWiperActuatorModel

20.7 V1.0 Platform Statistics

V1.0 Release Candidate statistics:

Metric	V1.0
Source files	265
TypeScript .ts files	219
React/TypeScript .tsx files	39
CSS files	7
Physical source lines	37,417
Approx. nonblank/non-comment source lines	31,135
Classes	124
Type aliases	256
Interfaces	12
Enums	1
Explicit TypeScript type constructs	269
Named function declarations	203
Canonical runtime events	102
Executable scenarios	24
Integrated BCM features	6
Vehicle packages	5
Vehicle colors	6

A concise publication statement is:

BCM Emulator V1.0 comprises more than 31,000 lines of source code across 265 source files, with 124 classes, 269 explicit TypeScript type constructs, 102 canonical runtime events, six integrated BCM features and 24 executable engineering scenarios.

20.8 Glossary

Term	Meaning in this guide
BCM	Body Control Module / body-control domain
EOS	Engineering Operating System
SDV	Software-Defined Vehicle
HPC	High-Performance Computer
ZC	Zone Controller
FuSa	Functional Safety
FTTI	Fault-Tolerant Time Interval
E2E	End-to-End protection
HAL	Hardware Abstraction Layer / modeled actuator-sensor boundary
Capability state	Whether a feature can provide its intended functionality
Operational state	Whether a feature is currently operating
Domain state	Feature-specific semantic/physical state
Dependency	A capability/provider relationship required by a consumer
Provider binding	The provider assigned to a dependency requirement
Quiescence	Controlled termination/invalidation of obsolete asynchronous runtime work

Term	Meaning in this guide
Reconstruction	Rebuilding current runtime truth from authoritative sources
Safe State	System-level safety reaction that remains controlled until recovery criteria are met

20.9 Version and Release Information

Product: BCM Emulator
Version: V1.0
Technical baseline: V1.0 Release Candidate
Release Candidate source: [src\(20260926-085701\).zip](#)
Guide formats: Word / PDF
Guide status: V1.0 publication edition
Public Preview URL: <https://bcm-emulator.netlify.app>

This publication edition is aligned with the hosted V1.0 Public Preview and the V1.0 Release Candidate baseline:

- 1. **Content and structure** — reviewed and frozen for the V1.0 publication.
- 2. **Visual publication layer** — hosted screenshots, conceptual diagrams and publication formatting integrated.

Future corrections should follow the Release Candidate discipline described in Section 18.10.

Explore the live BCM Emulator V1.0 Public Preview

[Open BCM Emulator Public Preview](#)

